



Escuela
Politécnica
Superior

Implementación de algoritmos de aprendizaje por refuerzo en robótica para la optimización de tareas autónomas



Grado en Ingeniería Robótica

Trabajo Fin de Grado

Autor:

Fernando López Verdejo

Tutor/es:

Francisco Antonio Pujol López

Mayo 2026



Universitat d'Alacant
Universidad de Alicante

Implementación de algoritmos de aprendizaje por refuerzo en robótica para la optimización de tareas autónomas

Autor

Fernando López Verdejo

Tutor/es

Francisco Antonio Pujol López

Departamento Tecnología Informática y Computación



Grado en Ingeniería Robótica



Escuela
Politécnica
Superior



Universitat d'Alacant
Universidad de Alicante

ALICANTE, Mayo 2026

Preámbulo

La realización de este trabajo de fin de grado surge del creciente interés, tanto académico como industrial, en dotar a la robótica móvil de una mayor autonomía e inteligencia. El propósito principal de este estudio ha sido explorar y validar la viabilidad de los algoritmos de aprendizaje por refuerzo profundo, específicamente la optimización de política próxima (PPO), aplicados a la navegación en entornos interiores complejos sin el uso de mapas pre-definidos.

El alcance de este proyecto abarca desde el diseño de los entornos de simulación en Gazebo hasta el entrenamiento, validación y análisis de convergencia de una arquitectura de control híbrida para el robot TurtleBot3. Este trabajo no solo consolida los conocimientos adquiridos durante la carrera, sino que abre la puerta a futuras implementaciones en entornos físicos reales.

Agradecimientos

Quiero expresar mi más sincero agradecimiento, a mi familia, por su apoyo incondicional, su comprensión en los momentos de mayor exigencia y por darme siempre el impulso necesario para seguir adelante. Sin su confianza y respaldo diario, alcanzar esta meta habría sido imposible.

A mis amigos y compañeros de grado, con quienes he compartido incontables horas de estudio, prácticas de laboratorio, debates y apoyo mutuo a lo largo de todos estos años universitarios. El camino ha sido mucho más enriquecedor gracias a vosotros.

A mi tutor, Paco, por su dedicación, paciencia y guía experta durante todo el desarrollo de este trabajo de fin de grado. Sus acertados consejos, su ayuda constante y su visión técnica han sido fundamentales para enfocar el proyecto, consolidar el trabajo realizado y llevarlo a buen puerto.

Finalmente, me gustaría agradecer a la Universidad de Alicante y, en especial, a todo el profesorado de la Escuela Politécnica Superior que ha contribuido de manera directa e indirecta a mi formación académica, personal y profesional.

*"La tarea no es tanto ver lo que nadie ha visto todavía,
sino pensar lo que nadie ha pensado todavía sobre aquello que todos ven."*

Erwin Schrödinger
Premio Nobel de Física, 1933

Índice general

1	Introducción	1
1.1	Estructura de la memoria	2
2	Marco Teórico	5
2.1	Tecnologías de robótica y simulación	5
2.1.1	El estándar ROS 2 y el middleware DDS	5
2.1.2	Simulación física y la brecha sim-to-real	5
2.2	Robótica móvil: modelado, localización y navegación clásica	6
2.2.1	Modelado cinemático	6
2.2.2	Modelado dinámico	7
2.2.3	Localización y mapeo probabilístico	7
2.2.4	Planificación global y local	8
2.2.5	Limitaciones clásicas: El problema del freezing robot	9
2.3	Inteligencia artificial y aprendizaje profundo	10
2.3.1	Redes neuronales y retropropagación	10
2.4	Aprendizaje por refuerzo (RL)	11
2.4.1	Elementos del aprendizaje por refuerzo	11
2.4.2	Procesos de decisión de Markov (MDP)	12
2.4.3	Funciones de valor y ecuación de Bellman	12
2.4.4	Taxonomía y clasificaciones del RL	13
2.4.5	Arquitectura actor-crítico y PPO	13
2.5	Aplicación del DRL a la navegación móvil	15
3	Objetivos	17
3.1	Objetivo general	17
3.2	Objetivos específicos	17
4	Metodología	19
4.1	Herramientas y librerías de <i>software</i>	19
4.1.1	Gymnasium (OpenAI Gym)	19
4.1.2	Stable-Baselines3 (SB3)	19
4.2	Selección y justificación del algoritmo	20
4.3	Configuración de la plataforma y ecosistema virtual	20
4.3.1	Plataforma robótica: TurtleBot3 Burger	20
4.3.2	Topología de comunicaciones en ROS 2	21
4.3.3	Integración del <i>framework</i> neuronal e hiperparámetros	21
4.4	Formulación matemática: Proceso de decisión de Markov (MDP)	22
4.4.1	Espacio de observación ($\mathcal{S}$): Discretización del <i>lidar</i>	22
4.4.2	Espacio de acciones ($\mathcal{A}$)	23

4.4.3	Función de diseño de la recompensa	23
4.5	Estrategia de aprendizaje en dos fases	23
4.5.1	Mecanismo de reseteo del entorno	24
4.6	Diseño de la arquitectura híbrida	24
4.6.1	Sistema de recuperación	25
4.7	Interfaz humano-robot (HRI)	25
5	Desarrollo	27
5.1	Arquitectura del sistema y flujo de datos en ROS 2	27
5.2	Despliegue del plan de entrenamiento	28
5.2.1	Fase 1: Entrenamiento base	28
5.2.2	Fase 2: Entrenamiento en entorno complejo	28
5.3	Implementación del módulo de percepción y recompensa	29
5.3.1	Submuestreo <i>lidar</i> mediante <i>min-pooling</i>	29
5.3.2	Codificación vectorizada de la recompensa	30
5.4	Programación de la arquitectura híbrida	31
5.4.1	Implementación del sistema de recuperación (<i>watchdog</i>)	32
5.4.2	Maniobras de recuperación y planificación vectorial	32
5.5	Evaluación ante un obstáculo dinámico simulado	33
5.6	Desarrollo de la interfaz gráfica	34
5.6.1	Sincronización de memoria	35
6	Resultados	37
6.1	Análisis de la percepción sensorial	37
6.2	Análisis de la evolución del entrenamiento PPO	37
6.2.1	Convergencia de la política: Recompensa y supervivencia	38
6.2.2	Exploración vs. explotación: Análisis de entropía	39
6.2.3	Estabilidad predictiva: Varianza y error de la red crítica	40
6.2.4	Rendimiento computacional y eficiencia temporal	40
6.3	Evaluación cuantitativa de la arquitectura híbrida	41
6.4	Validación en misión secuencial	42
6.5	Análisis de eficiencia logística y consumo simulado	42
7	Conclusiones	45
7.1	Conclusiones finales	45
7.2	Líneas de trabajo futuro	46
	Bibliografía	47

Índice de figuras

1.1	Tipos de robots móviles autónomos operando en entornos logísticos. . . .	1
2.1	Representación del modelo de cinemática diferencial.	7
2.2	Ejemplo de un mapa de rejilla de ocupación generado mediante técnicas SLAM.	8
2.3	Representación gráfica de las trayectorias proyectadas por un planificador local basado en la ventana dinámica ante la presencia de un obstáculo. . . .	9
2.4	Esquema del problema del <i>freezing robot</i> , donde la disposición de los obstáculos restringe temporalmente el avance planificado.	10
2.5	Modelo matemático de una red neuronal.	11
2.6	El agente observa el estado actual (S_t), ejecuta una acción (A_t) y recibe del entorno el estado resultante junto con una señal escalar de recompensa (R_t). . . .	12
2.7	El crítico evalúa la recompensa para calcular el error de diferencia temporal ($Error \delta$), utilizado para guiar la optimización de la política del actor.	15
4.1	Componentes de <i>hardware</i> del TurtleBot3 Burger.	20
4.2	Esquema conceptual de las interfaces y capas de comunicación en la arquitectura ROS 2.	21
4.3	Reducción de la dimensionalidad del láser mediante agrupación por sectores. . . .	22
5.1	Diagrama de flujo de datos y paso de mensajes entre nodos en la arquitectura ROS 2 implementada.	28
5.2	Escenarios de entrenamiento en Gazebo: Entorno residencial (izq.) para la fase 2 y entorno inicial (der.) para la fase 1.	29
5.3	Esquema lógico y matemático de los componentes de la función de recompensa implementada en el código.	30
5.4	Esquema lógico implementado para la conmutación de capas del controlador híbrido.	31
5.5	Visualización de la ruta programada utilizando <i>waypoints</i> de aproximación. . . .	33
5.6	Interfaz gráfica operativa programada para la gestión y monitorización. . . .	34
6.1	Comparativa de la duración media y recompensa del agente.	38
6.2	Comparativa de la entropía. Arriba: Oficina. Abajo: Casa.	39
6.3	Comparativa de la varianza y el error predictivo.	40

Índice de tablas

6.1	Tabla comparativa de las métricas de la red neuronal en las 2 fases. . . .	38
6.2	Evaluación simulada basada en 20 pruebas con las 3 métricas de rendimiento.	41
6.3	Desglose de tiempos y estimación de consumo energético en simulación. .	42

Índice de Códigos

5.1	Algoritmo de agrupación sectorial LiDAR implementado en el <i>callback</i> del entorno.	29
-----	---	----

1 Introducción

La robótica móvil ha experimentado una evolución constante en la última década, diversificando sus aplicaciones desde entornos industriales estructurados hacia espacios compartidos con seres humanos. Este cambio, impulsado por el incremento en la capacidad de cómputo embebido y la disponibilidad de sensores de percepción como los sistemas *lidar* y las cámaras de profundidad, ha fomentado el desarrollo de robots de servicio. Sectores como la logística de última milla, la asistencia hospitalaria o el mantenimiento residencial requieren plataformas autónomas capaces de navegar en escenarios caracterizados por el dinamismo y la incertidumbre [1].

Este contexto demanda la integración de algoritmos que favorezcan la seguridad operativa. A diferencia del desplazamiento predecible en entornos automatizados cerrados, el sistema robótico actual debe ser capaz de reaccionar ante movimientos de humanos y cambios en la topología del entorno, velando por la integridad de las personas, del mobiliario y de la propia plataforma.



Figura 1.1: Tipos de robots móviles autónomos operando en entornos logísticos.

Fuente: [2]

Tradicionalmente, el problema de la navegación autónoma se ha resuelto mediante una arquitectura de control jerárquica: un planificador global calcula la trayectoria óptima sobre un mapa estático, mientras que un planificador local genera los comandos cinemáticos para evitar obstáculos [3]. El uso de algoritmos de búsqueda en grafos, como el de Dijkstra, constituye un estándar industrial extendido para el cálculo de trayectorias globales [4]. Por su parte, para la evasión reactiva, el enfoque de ventana dinámica (DWA) es una de las soluciones más utilizadas por su eficiencia computacional [5]. No obstante, la conjunción de estas técnicas

puede presentar limitaciones operativas cuando aumenta la densidad del tráfico peatonal.

Una de las limitaciones documentadas de estos esquemas geométricos es el denominado problema del robot congelado (*freezing robot problem*). Este fenómeno ocurre cuando los algoritmos de planificación local determinan que cualquier movimiento cinemático proyectado resulta en una colisión inminente. Al carecer de una heurística para predecir trayectorias de obstáculos dinámicos, el planificador puede detener la marcha de la plataforma de forma prolongada. Esta situación motiva el interés por investigar arquitecturas que permitan maniobras de esquivas más fluidas.

Para dotar a la plataforma de una mayor capacidad de respuesta, esta investigación plantea el uso de técnicas de aprendizaje por refuerzo profundo (DRL) [6]. A diferencia de la programación basada en reglas rígidas, el DRL permite que el agente robótico desarrolle su propia política de control mediante la interacción iterativa con el entorno, optimizando la toma de decisiones a través de una señal de recompensa escalar [7].

Dentro del ecosistema del aprendizaje profundo, se ha seleccionado el algoritmo *proximal policy optimization* (PPO) [8]. PPO ofrece una notable estabilidad durante el proceso de optimización y permite operar en espacios de acción continuos. Esta característica es relevante en la cinemática diferencial robótica, ya que facilita ajustes progresivos en las velocidades de los motores. El núcleo de este proyecto propone una arquitectura híbrida que combina la eficiencia de la navegación clásica para rutas extensas con la capacidad reactiva de una red neuronal ante obstáculos dinámicos.

A nivel de implementación, el sistema se ha validado en un entorno simulado utilizando la plataforma TurtleBot3 Burger y su escáner *lidar* de 360 grados [9]. Para la gestión de procesos y comunicaciones se emplea el *middleware* ROS 2 Humble [10]. El entrenamiento de la red neuronal se ha llevado a cabo en el simulador Gazebo [11], aplicando técnicas de regularización para sentar las bases de una futura transferencia *sim-to-real* [12]. Como pilar organizativo, el sistema se integra en una interfaz gráfica de usuario (GUI) desarrollada en Python para la monitorización de las misiones simuladas.

1.1 Estructura de la memoria

Para facilitar la comprensión del trabajo realizado, el documento se estructura de la siguiente manera:

- **Capítulo 2. Marco teórico:** Describe los fundamentos de la cinemática robótica, el estándar ROS 2 y las bases matemáticas del aprendizaje por refuerzo.
 - **Capítulo 3. Objetivos:** Define la meta de la arquitectura híbrida y los hitos técnicos necesarios.
 - **Capítulo 4. Metodología:** Justifica las decisiones de diseño, la formulación de la recompensa y la estrategia de aprendizaje en dos fases.
 - **Capítulo 5. Desarrollo:** Detalla el flujo de programación del entorno mediante Gymnasium, la configuración de PPO y el diseño del sistema watchdog.
-

- **Capítulo 6. Resultados:** Presenta el análisis del entrenamiento y la tasa de éxito obtenida en simulación frente a algoritmos clásicos.
 - **Capítulo 7. Conclusiones:** Sintetiza los resultados alcanzados y propone líneas de mejora para el despliegue en hardware real.
-

2 Marco Teórico

Para comprender las decisiones arquitectónicas de este proyecto, es necesario establecer primero un marco teórico. Este capítulo detalla los fundamentos teóricos necesarios para comprender el aprendizaje por refuerzo, partiendo de la inteligencia artificial y el aprendizaje profundo. Se analiza la evolución de los sistemas basados en reglas hacia modelos capaces de aprender de la experiencia, concluyendo con la integración de estos algoritmos en la robótica móvil [6].

2.1 Tecnologías de robótica y simulación

El desarrollo actual de *software* robótico se basa en la abstracción del *hardware* mediante *middleware* estandarizado y el uso de motores físicos de simulación [3].

2.1.1 El estándar ROS 2 y el middleware DDS

El sistema operativo de robots (ROS) actúa como un *middleware* que proporciona abstracción del *hardware*, control de bajo nivel y un sistema de paso de mensajes entre procesos. La evolución hacia ROS 2 Humble introdujo cambios estructurales para proporcionar características de tiempo real y seguridad en entornos industriales [10].

La arquitectura de comunicaciones de ROS 2 se fundamenta en el estándar DDS (*data distribution service*). Esta adopción elimina la dependencia de un nodo maestro centralizado, buscando evitar cuellos de botella y fallos únicos en el sistema. El ecosistema opera bajo un grafo distribuido compuesto por:

- **Nodos:** Unidades de procesamiento concurrente. En un robot complejo, la visión, la odometría y el control residen en nodos independientes.
- **Tópicos:** Canales de publicación/suscripción asíncronos.
- **Calidad de servicio (QoS):** DDS permite definir perfiles de fiabilidad. Por ejemplo, para sensores de alta frecuencia como el *lidar*, se utiliza frecuentemente el perfil *best effort*, que prioriza la baja latencia (entregando el dato más reciente) sobre la fiabilidad de entrega.

2.1.2 Simulación física y la brecha sim-to-real

El entrenamiento de redes neuronales requiere un volumen de interacciones que resulta difícil de alcanzar exclusivamente con *hardware* físico. Simuladores 3D como Gazebo permiten abordar este problema integrando motores de dinámica de cuerpos rígidos (como el *open*

dynamics engine, ODE), que calculan fuerzas, colisiones, inercias y gravedad aplicando las leyes físicas en tiempo real [11].

Un aspecto relevante en este ámbito es la brecha *sim-to-real gap*. Los simuladores son aproximaciones matemáticas que no replican con exactitud el desgaste mecánico o las imperfecciones del mundo real. Para mitigar esta desviación en simulaciones orientadas a un posterior despliegue físico, se suelen aplicar técnicas de regularización como la inyección de ruido sensorial y la aleatorización del dominio (*domain randomization*), con el fin de que la red neuronal aprenda políticas más robustas frente a variaciones del entorno [12].

2.2 Robótica móvil: modelado, localización y navegación clásica

Para dimensionar el impacto del aprendizaje por refuerzo en la robótica, es preciso comprender los enfoques tradicionales de control y navegación, los cuales se basan en el conocimiento explícito del entorno.

2.2.1 Modelado cinemático

La interacción física de un vehículo con su entorno para conseguir un desplazamiento se denomina locomoción. Para que un vehículo con ruedas cambie de orientación, su movimiento debe pivotar sobre un punto geométrico conocido como centro de curvatura instantáneo (CCI) [1].

El modelado cinemático describe este movimiento centrándose en su geometría y restricciones. De manera general, las configuraciones con ruedas imponen restricciones no holonómicas; es decir, el robot no puede alcanzar todas las posiciones de su espacio de trabajo de manera directa (por ejemplo, desplazarse lateralmente pura y físicamente), lo que le obliga a maniobrar [3]. Destacan dos configuraciones principales:

- **Modelo Ackerman:** Utilizado habitualmente en automoción. Las ruedas delanteras son direccionales y giran con ángulos independientes (δ_i, δ_o) para hacer coincidir sus ejes perpendiculares en el CCI.
- **Modelo diferencial:** Es el utilizado por la plataforma TurtleBot3 Burger [9]. El movimiento es controlado por la diferencia de velocidad entre dos ruedas motrices coaxiales independientes, requiriendo ruedas locas (*caster*) pasivas para mantener el equilibrio.

Matemáticamente en el modelo diferencial, la pose global del robot en el instante t se define por el vector $[x, y, \theta]^T$. Las ecuaciones de estado continuo que rigen la evolución de esta pose dependen de la velocidad lineal (v) y angular (ω) aplicadas al centroide del chasis [1]:

$$\dot{x} = v \cos(\theta), \quad \dot{y} = v \sin(\theta), \quad \dot{\theta} = \omega \quad (2.1)$$

A su vez, estas velocidades de control se calculan a partir de las velocidades angulares de la rueda derecha (ω_R) e izquierda (ω_L). Siendo r el radio de las ruedas y L la distancia de separación entre ambas (vía), el modelo cinemático diferencial completo se define de la siguiente manera [1]:

$$v = \frac{r}{2}(\omega_R + \omega_L), \quad \omega = \frac{r}{L}(\omega_R - \omega_L) \quad (2.2)$$

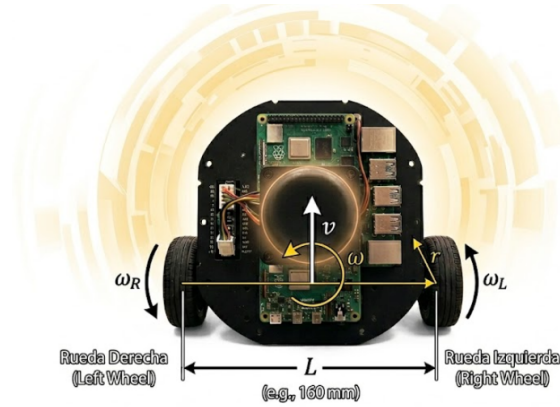


Figura 2.1: Representación del modelo de cinemática diferencial.

Fuente: Adaptado de [9].

2.2.2 Modelado dinámico

A diferencia del modelado cinemático, que se enfoca en la geometría del movimiento sin considerar las fuerzas involucradas, el modelado dinámico describe la relación entre las fuerzas y torques aplicados al sistema y el movimiento resultante [3]. Esto permite una representación más ajustada a la física al incluir efectos como la inercia y la fricción.

En aplicaciones reales y en simuladores físicos como Gazebo, los robots móviles están sujetos a restricciones que afectan su desempeño, como la aceleración limitada por la potencia del motor o la influencia del rozamiento de las ruedas con el suelo [3].

El movimiento del robot puede describirse mediante las ecuaciones de Newton-Euler, que relacionan fuerzas (F) y torques (τ) con las aceleraciones respectivas [3]:

$$F = m \cdot a \quad (2.3)$$

$$\tau = I \cdot \alpha \quad (2.4)$$

donde m es la masa, a la aceleración lineal, I el momento de inercia respecto del eje de giro y α la aceleración angular. En este trabajo, el motor de dinámicas del simulador gestiona estos cálculos físicos para modelar la interacción del agente con el entorno.

2.2.3 Localización y mapeo probabilístico

La navegación autónoma clásica requiere que el robot estime su pose en el mundo. Dado que la odometría acumula errores por el deslizamiento de las ruedas, se suele recurrir a métodos probabilísticos como la localización de Markov y al filtro de Bayes [13].

El problema de localización y mapeo simultáneo (SLAM) se aborda clásicamente mediante algoritmos que estiman conjuntamente la trayectoria del robot y un mapa de ocupación de rejilla [13]. En entornos simulados donde la pose exacta es accesible mediante la telemetría del simulador, estas técnicas de estimación probabilística pueden omitirse para simplificar el despliegue del agente.



Figura 2.2: Ejemplo de un mapa de rejilla de ocupación generado mediante técnicas SLAM.

Fuente: [13]

2.2.4 Planificación global y local

La arquitectura tradicional divide el cálculo de trayectorias en dos subsistemas [1]:

1. Planificación global de trayectorias: El objetivo es encontrar una ruta en el mapa a través del espacio libre. Para facilitar los cálculos, se suele dilatar el tamaño de los obstáculos con un radio igual al del robot y se genera un grafo de búsqueda. Algunos de los algoritmos más comunes son:

- **Dijkstra:** Explora iterativamente los nodos con menor distancia acumulada hasta alcanzar el destino [4].
- **A*:** Introduce una función heurística para optimizar la búsqueda [14]. Evalúa cada nodo minimizando $f(n) = g(n) + h(n)$, donde $g(n)$ es el coste real acumulado y $h(n)$ es la estimación heurística.
- **D*:** Variante de A* diseñada para entornos parcialmente dinámicos, que permite re-planificar rutas recalculando solo las zonas afectadas.
- **Técnicas probabilísticas (PRM y RRT):** En mapas extensos, algoritmos como *rapidly-exploring random tree* (RRT) expanden árboles de búsqueda de manera aleatoria, encontrando soluciones en espacios de alta dimensionalidad.

2. Planificación local: Se encarga de evitar obstáculos dinámicos no presentes en el mapa original:

- **Campos de fuerzas (VFF y VFH):** Asignan fuerzas de atracción hacia la meta y de repulsión hacia los obstáculos. El *vector field histogram* (VFH) utiliza un histograma polar para decidir la dirección libre óptima [15].

- **Ventana dinámica (DWA):** Proyecta las trayectorias circulares que el robot podría trazar en un instante Δt dadas sus limitaciones dinámicas. Luego, evalúa esas velocidades (v, ω) mediante una función objetivo que pondera el avance hacia la meta, la distancia a los obstáculos y la velocidad de traslación [16].

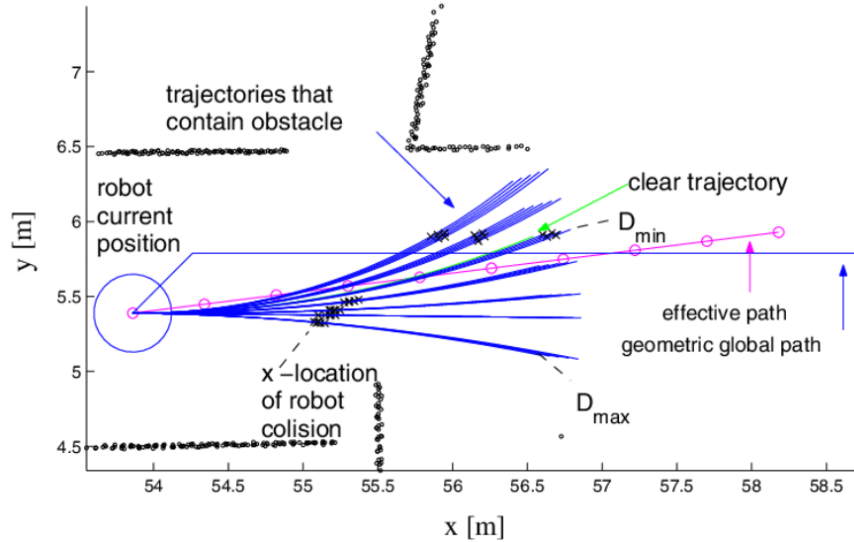


Figura 2.3: Representación gráfica de las trayectorias proyectadas por un planificador local basado en la ventana dinámica ante la presencia de un obstáculo.

Fuente: Adaptado de [5].

2.2.5 Limitaciones clásicas: El problema del freezing robot

A pesar de su extendido uso, planificadores geométricos como DWA pueden presentar limitaciones operativas en entornos dinámicos. Uno de los problemas teóricos documentados en este ámbito es el fenómeno conocido como el *freezing robot* [5].

Matemáticamente, DWA proyecta posibles velocidades futuras y descarta aquellas que resulten en colisión. En escenarios donde la densidad de obstáculos (como peatones) aumenta, el espacio de velocidades seguras evaluado por el algoritmo puede reducirse a un conjunto vacío. Al no encontrar trayectorias libres de colisión en su ventana de búsqueda, el algoritmo establece los comandos de velocidad a cero [5].

Esta situación puede provocar que el robot detenga su marcha a la espera de que el entorno se despeje, lo que ha fomentado la investigación en arquitecturas reactivas basadas en inteligencia artificial [6].

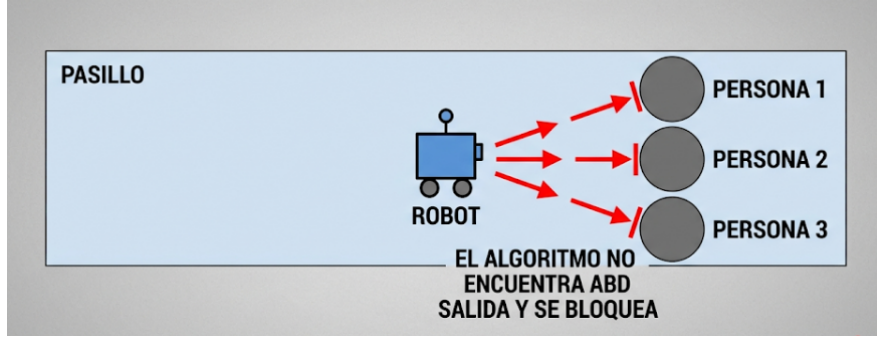


Figura 2.4: Esquema del problema del *freezing robot*, donde la disposición de los obstáculos restringe temporalmente el avance planificado.

2.3 Inteligencia artificial y aprendizaje profundo

El aprendizaje automático busca permitir que los sistemas computacionales identifiquen patrones directamente a partir de datos, ofreciendo alternativas para gestionar la incertidumbre de escenarios complejos [6]. Se distinguen tres paradigmas principales [7]:

- **Aprendizaje supervisado:** El modelo se entrena con pares de datos de entrada y salida conocida (etiquetas), buscando generalizar la función subyacente.
- **Aprendizaje no supervisado:** El sistema busca estructuras ocultas en datos sin etiquetar.
- **Aprendizaje por refuerzo:** Un agente ajusta sus acciones interactuando con un entorno a través del método de ensayo y error, guiado por una señal de recompensa.

2.3.1 Redes neuronales y retropropagación

Las redes neuronales artificiales se estructuran mediante nodos (neuronas) organizados en capas. Cada conexión posee un peso (w) y cada neurona un sesgo (b). La salida se define matemáticamente aplicando una función de activación no lineal [17]:

$$z = f \left(\sum_{i=1}^n w_i x_i + b \right) \quad (2.5)$$

El entrenamiento de estos modelos se fundamenta en el algoritmo de retropropagación, que calcula el gradiente de la función de error respecto a cada peso de la red. Algoritmos de optimización (como Adam) ajustan posteriormente dichos pesos. Cuando se emplean múltiples capas ocultas, el enfoque se denomina aprendizaje profundo (*deep learning*) [18].

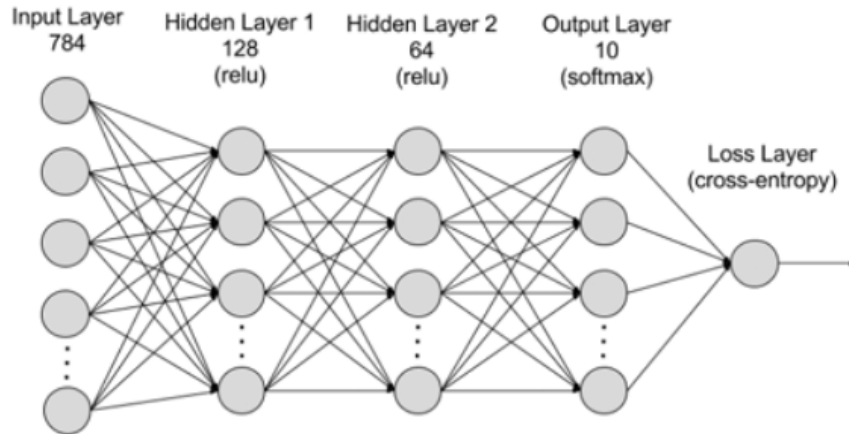


Figura 2.5: Modelo matemático de una red neuronal.

Fuente: [17]

2.4 Aprendizaje por refuerzo (RL)

El RL plantea el aprendizaje autónomo mediante la optimización de una recompensa acumulada [7]. La integración de redes neuronales en este paradigma originó el aprendizaje por refuerzo profundo (DRL) [19], permitiendo operar en espacios de estados más amplios y continuos.

2.4.1 Elementos del aprendizaje por refuerzo

El proceso de aprendizaje se formula mediante la interacción continua de los siguientes elementos [7]:

- **Agente y entorno:** El agente es la entidad que toma decisiones dentro de un espacio denominado entorno. El agente debe mantener un balance entre ejecutar acciones aleatorias para obtener nueva información (exploración) y utilizar el conocimiento ya adquirido (explotación).
- **Estados (s_t) y observaciones (o_t):** En aplicaciones de robótica real o simulada, la información del estado completo suele estar limitada por las capacidades de los sensores. Esta descripción parcial recibe el nombre de observación.
- **Acciones (a_t):** El conjunto de decisiones posibles puede ser discreto o continuo. En cinemática diferencial, se suele operar en un espacio de acciones continuo para modular la velocidad.
- **Recompensa (r_t):** Señal escalar que retroalimenta al agente. El objetivo matemático se define a través de la función de retorno esperado ($R(\tau)$), que evalúa la trayectoria

descontando los valores futuros mediante el factor $\gamma \in [0, 1]$:

$$R(\tau) = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (2.6)$$

- **Política (π):** Función que mapea los estados a las acciones correspondientes, pudiendo adoptar un enfoque determinista o estocástico.

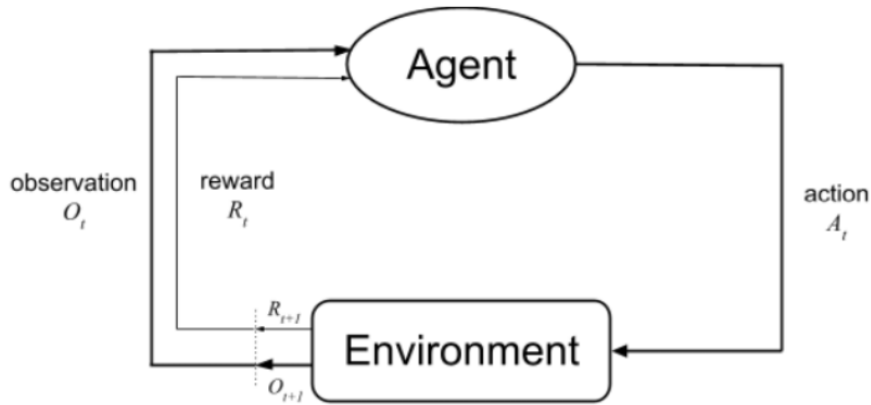


Figura 2.6: El agente observa el estado actual (S_t), ejecuta una acción (A_t) y recibe del entorno el estado resultante junto con una señal escalar de recompensa (R_t).

Fuente: Adaptado de [6].

2.4.2 Procesos de decisión de Markov (MDP)

El marco teórico para formular estos problemas de control son los procesos de decisión de Markov (MDP) [7]. Operan bajo la propiedad de Markov, donde la transición al siguiente estado depende solo del estado y acción actuales. Se definen por la tupla $\langle S, A, P, R, \gamma \rangle$, donde $P(s'|s, a)$ representa la distribución de probabilidad de transición.

2.4.3 Funciones de valor y ecuación de Bellman

Para evaluar la calidad de una política π , se utilizan la función de valor de estado $v_\pi(s)$ y la función de valor estado-acción $q_\pi(s, a)$ [7]:

$$v_\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s \right] \quad (2.7)$$

$$q_\pi(s, a) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t = s, A_t = a \right] \quad (2.8)$$

Se resuelven recursivamente mediante la ecuación de Bellman, descomponiendo el valor de un estado en la recompensa inmediata más el valor esperado del siguiente estado [7]:

$$v_{\pi}(s) = \sum_a \pi(a|s) \sum_{s',r} p(s',r|s,a) [r + \gamma v_{\pi}(s')] \quad (2.9)$$

2.4.4 Taxonomía y clasificaciones del RL

Los algoritmos de RL pueden clasificarse según su enfoque [7]:

1. Según el conocimiento del entorno:

- **Basado en modelo (*model-based*):** El agente tiene acceso a las dinámicas de transición del entorno, permitiendo planificar a través de la ecuación de Bellman.
- **Libre de modelo (*model-free*):** El agente desconoce las dinámicas del entorno y aprende mediante la recolección de experiencias empíricas.

2. Según el muestreo:

- ***On-policy*:** El agente evalúa y mejora la misma política que utiliza para tomar decisiones.
- ***Off-policy*:** El agente disocia la política de comportamiento (exploración) de la política objetivo, pudiendo reaprovechar experiencias pasadas.

3. Según el objetivo de optimización:

- ***Value-based*:** Optimizan la función de valor para inferir posteriormente la acción óptima (ej. Q-Learning).
- ***Policy-based*:** Parametrizan y optimizan directamente la política de control.
- ***Actor-critic*:** Combinan ambos enfoques; un módulo actor selecciona las acciones y un módulo crítico estima su valor.

2.4.5 Arquitectura actor-crítico y PPO

Para gestionar espacios de control continuo (como las velocidades de un robot diferencial), los enfoques actor-crítico resultan de gran utilidad al desplegar dos redes neuronales que operan conjuntamente [7]:

El actor ($\pi_{\theta}(a_t|s_t)$): Parametrizado por pesos θ , genera la distribución de probabilidad de las acciones basándose en el estado observado.

El crítico ($V_{\phi}(s_t)$): Parametrizado por pesos ϕ , evalúa el estado actual para estimar el retorno escalar esperado, buscando reducir la varianza durante el entrenamiento.

El aprendizaje del actor está guiado por el error de diferencia temporal (δ_t) [7]:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (2.10)$$

Este error sirve como base para calcular la función de ventaja (A_t), que cuantifica si una acción fue mejor de lo promediamente esperado. En implementaciones modernas, se suele recurrir a la estimación de ventaja generalizada (GAE) para equilibrar el sesgo y la varianza [8]:

$$\hat{A}_t^{GAE(\gamma, \lambda)} = \sum_{l=0}^{\infty} (\gamma \lambda)^l \delta_{t+l} \quad (2.11)$$

El algoritmo *proximal policy optimization* (PPO) plantea una actualización controlada de la política evaluando el ratio $r_t(\theta)$ entre la política nueva y la iteración anterior [8]:

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \quad (2.12)$$

Una de las características principales de PPO es su función de pérdida subrogada con recorte (*clipping*), diseñada para buscar la estabilidad en el aprendizaje [8]:

$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[\min \left(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right] \quad (2.13)$$

El hiperparámetro ϵ delimita una región de confianza. La operación *clip* restringe los cambios abruptos en la política: si la ventaja es positiva, se acota la actualización al límite superior $1 + \epsilon$ para intentar evitar un sobreajuste; si la ventaja es negativa, se limita la penalización para prevenir inestabilidades en el conocimiento ya adquirido.

La función objetivo general de PPO combina este término con el error de la red crítica y un factor de entropía, cuyo propósito es incentivar la exploración del entorno [8]:

$$L_t^{TOTAL}(\theta) = \hat{\mathbb{E}}_t \left[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t) \right] \quad (2.14)$$

En términos de implementación de *software*, se emplea Gymnasium para estandarizar la interacción entre el entorno de simulación y el agente [20], y la librería Stable-Baselines3 para el despliegue del algoritmo PPO [21].

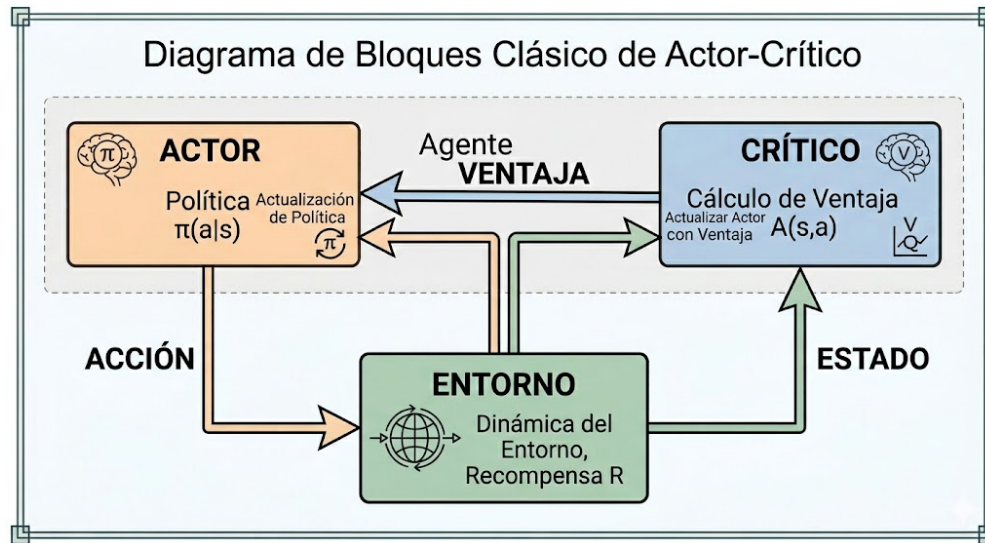


Figura 2.7: El crítico evalúa la recompensa para calcular el error de diferencia temporal (*Error δ*), utilizado para guiar la optimización de la política del actor.

2.5 Aplicación del DRL a la navegación móvil

La arquitectura tradicional suele depender de módulos serializados (localización $\rightarrow$ planificación global $\rightarrow$ control local), lo cual puede introducir limitaciones ante imprevistos si el error de estimación de la pose se propaga en cascada [6].

La exploración de enfoques basados en aprendizaje por refuerzo profundo (DRL) aporta características de diseño que se proponen como alternativa a algunos modelos puramente geométricos [6]:

1. **Navegación *mapless* (*end-to-end*):** Al mapear las lecturas en crudo del *lidar* a comandos de velocidad de bajo nivel, se busca reducir la dependencia estricta de costosas triangulaciones geométricas en tiempo real [22].
2. **Tolerancia a variaciones dinámicas:** Modelando estocásticamente el entorno durante la fase de simulación, las políticas pueden ayudar a asimilar el ruido sensorial de forma más estable ante el dinamismo del entorno.
3. **Uso de simulación física:** El uso de simuladores facilita la obtención de políticas que, acompañadas de las técnicas de regularización metodológicamente adecuadas, pueden servir como conocimiento base para plantear en un futuro un eventual despliegue *sim-to-real* en chasis físicos, minimizando así riesgos operativos.

3 Objetivos

El desarrollo de un sistema de navegación autónoma en entornos dinámicos requiere la integración de diversas áreas tecnológicas: desde el control cinemático y la percepción sensorial, hasta el entrenamiento de redes neuronales y el diseño de interfaces de usuario.

Para abordar este desarrollo de forma estructurada, la presente memoria se ha organizado en torno a un objetivo principal, del cual se deriva una serie de metas específicas que sirven como hoja de ruta para el proyecto.

3.1 Objetivo general

El objetivo principal de este proyecto es diseñar, implementar y validar en un entorno simulado un sistema de navegación autónoma para la plataforma robótica diferencial TurtleBot3, orientado a mitigar las limitaciones de los algoritmos clásicos de planificación geométrica en entornos dinámicos.

Para lograrlo, se propone el desarrollo de una arquitectura híbrida que integre técnicas de aprendizaje por refuerzo profundo. Este sistema está enfocado a permitir al robot ejecutar misiones logísticas en interiores simulados de forma continua, buscando una evasión de obstáculos reactiva, todo ello monitorizado a través de una interfaz gráfica de usuario.

3.2 Objetivos específicos

Para alcanzar el objetivo general, el trabajo se ha fragmentado en siete objetivos específicos que abarcan las distintas fases de desarrollo del *software*:

- **1. Diseño de entornos de simulación progresivos:** Construir y configurar escenarios tridimensionales de distinta complejidad topológica en Gazebo. El fin es establecer un plan de aprendizaje en dos fases que ayude a reducir el sobreajuste de la red neuronal y fomente su capacidad de generalización espacial.
- **2. Procesamiento sensorial y reducción de dimensionalidad:** Desarrollar un módulo capaz de procesar la nube de puntos del sensor *lidar* mediante submuestreo espacial. Esto permitirá crear un vector de observación compacto que facilite inferencias neuronales a alta frecuencia (20 Hz) minimizando la carga computacional.
- **3. Diseño de la función de recompensa:** Formular y calibrar una función multiobjetivo para guiar el entrenamiento del algoritmo PPO. Esta función buscará penalizar las colisiones e incentivar maniobras eficientes, con el propósito de mitigar problemas clásicos como el *freezing robot* en pasillos estrechos.

- **4. Validación frente a un obstáculo dinámico:** Integrar y simular actores móviles independientes en el entorno virtual, emulando el comportamiento de peatones humanos. Esto permitirá someter a prueba la política neuronal obtenida y evaluar su capacidad de evasión en tiempo real.
 - **5. Implementación de la arquitectura híbrida:** Diseñar la lógica de control que orqueste la navegación del robot. Se programará un sistema para conmutar dinámicamente entre el enrutamiento clásico por odometría para trayectos largos y la inferencia de la inteligencia artificial ante riesgos de colisión.
 - **6. Desarrollo de rutinas de recuperación:** Programar algoritmos de supervisión que monitoricen el estado del chasis para detectar bloqueos prolongados o caídas en mínimos locales, inyectando maniobras de rescate orientadas a favorecer la autonomía sostenida del sistema.
 - **7. Creación de una interfaz de supervisión asíncrona:** Desarrollar una aplicación de escritorio mediante arquitectura multihilo (*multithreading*). El objetivo es separar la carga gráfica de los procesos de ROS 2, permitiendo al usuario enviar misiones y visualizar la telemetría sin interferir en los cálculos críticos de control.
-

4 Metodología

El desarrollo de un sistema de navegación autónoma en entornos dinámicos plantea diversos retos a nivel algorítmico y de control. Por consiguiente, la metodología adoptada en este trabajo se lleva a cabo en un diseño de investigación experimental, iterativo y cuantitativo.

A diferencia de los enfoques heurísticos tradicionales, donde el comportamiento del robot se programa explícitamente mediante reglas geométricas, este proyecto requiere una metodología que permita al agente inferir su propia política de control a partir de la experiencia empírica. La realización de este trabajo será evaluada en su totalidad en un entorno de simulación, lo cual facilita la fase de entrenamiento masivo, alivia el desgaste de los componentes y permite establecer un modelo base para una futura transferencia a plataformas físicas.

4.1 Herramientas y librerías de *software*

Para la implementación se ha utilizado Python, un lenguaje ampliamente adoptado en el ámbito del aprendizaje por refuerzo debido a su ecosistema de *frameworks* de inteligencia artificial. La gestión de paquetes se orquestó mediante Conda, permitiendo la creación de entornos virtuales aislados para prevenir conflictos de dependencias entre ROS 2 y las librerías necesarias.

4.1.1 Gymnasium (OpenAI Gym)

Para la estructuración del entorno de aprendizaje, se empleó la librería Gymnasium. Esta herramienta proporciona una interfaz de programación de aplicaciones (API) estandarizada (`gym.Env`) para interactuar con agentes de aprendizaje por refuerzo. Su principal ventaja metodológica es que permite encapsular simuladores (como la integración ROS 2 y Gazebo) bajo métodos estructurados de inicialización (*reset*) y transición de estado (*step*), facilitando la prueba de diferentes arquitecturas sin alterar el control de bajo nivel del robot.

4.1.2 Stable-Baselines3 (SB3)

Para el entrenamiento del modelo, se integró Stable-Baselines3, una biblioteca de código abierto basada en PyTorch que provee implementaciones optimizadas de algoritmos DRL. SB3 fue seleccionada frente a otras alternativas por su integración nativa con herramientas de recolección de métricas como TensorBoard, lo cual facilita el análisis durante la fase de entrenamiento.

4.2 Selección y justificación del algoritmo

La biblioteca SB3 ofrece diversos algoritmos clasificados según el espacio de acciones que soportan. Dado que la locomoción de un robot móvil (velocidades lineal y angular) se modela de forma natural mediante un espacio de acciones continuo, algoritmos diseñados para espacios discretos, como DQN clásico, no resultan adecuados para este propósito.

Dentro de las opciones para espacios continuos, los algoritmos basados en la arquitectura actor-crítico destacan por su versatilidad. Para este trabajo, se evaluó teóricamente el uso de A2C frente a PPO (*proximal policy optimization*). Finalmente, la metodología se centró en PPO debido a su mecanismo de recorte (*clipping*), el cual limita las actualizaciones abruptas de la política y busca proporcionar una mayor estabilidad durante el entrenamiento en entornos con espacios estrechos y obstáculos.

4.3 Configuración de la plataforma y ecosistema virtual

La experimentación necesaria para el entrenamiento en aprendizaje por refuerzo profundo dificulta su ejecución directa sobre *hardware* físico en las etapas iniciales. Por ello, se estableció un ecosistema de simulación interconectado.

4.3.1 Plataforma robótica: TurtleBot3 Burger

El modelo utilizado se fundamenta en el TurtleBot3 Burger. Su elección se justifica por ser una plataforma académica estandarizada, con un sistema de tracción diferencial basado en dos motores y un sensor *lidar* LDS-01, el cual proporciona el escaneo de 360 grados necesario para construir el vector de observación de la red neuronal.

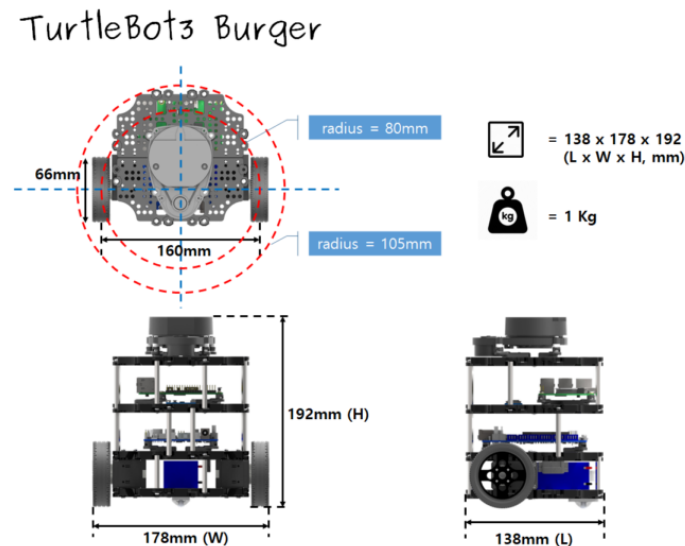


Figura 4.1: Componentes de *hardware* del TurtleBot3 Burger.

Fuente: [9]

4.3.2 Topología de comunicaciones en ROS 2

El ecosistema de desarrollo se orquestó sobre ROS 2 Humble. Para intentar garantizar que el nodo de inferencia neuronal reciba los flujos de datos con la menor latencia posible, la topología de red se configuró de la siguiente manera:

- **Topics y calidad de servicio (QoS):** El flujo de telemetría proveniente del sensor *lidar* (*topic /scan*) requiere un procesamiento ágil. Para mitigar retrasos por encolamiento, se configuró este *topic* con un perfil QoS *best effort* (*KEEP_LAST*, *depth=10*), que prioriza la inmediatez sobre la fiabilidad de entrega. El *topic /cmd_vel*, encargado de los comandos de actuación, operó bajo un perfil fiable estándar. La posición global se obtuvo del *topic /odom*, procesando los cuaterniones espaciales a ángulos de Euler.
- **Servicios asíncronos:** Se implementaron llamadas a servicios del sistema (como */reset_simulation*) para reiniciar las dinámicas del simulador Gazebo tras cada episodio terminal.

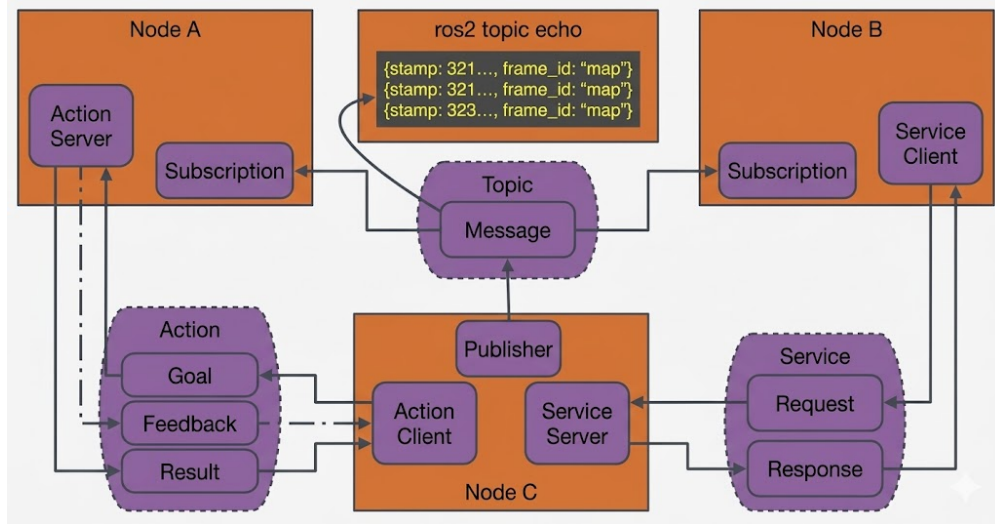


Figura 4.2: Esquema conceptual de las interfaces y capas de comunicación en la arquitectura ROS 2.

4.3.3 Integración del *framework* neuronal e hiperparámetros

El control de navegación se encapsuló en una clase `TurtleBotEnv`. Esta estructura permitió evaluar los ciclos de control en intervalos de 0.05 segundos (*timesteps*), equivalentes a una frecuencia de inferencia teórica de 20 Hz.

Para las redes del actor y del crítico, se definió una arquitectura de perceptrón multicapa (MLP). Los hiperparámetros de PPO se configuraron buscando un equilibrio entre exploración y convergencia:

- **Tasa de aprendizaje (*Learning Rate*):** 3×10^{-4} .

- **Factor de descuento (γ):** 0.99.
- **Parámetro de recorte PPO (ϵ):** 0.2, para mantener la actualización en una región de confianza controlada.
- **Coefficiente de entropía:** 0.0, para favorecer que la política converja hacia la explotación.

4.4 Formulación matemática: Proceso de decisión de Markov (MDP)

4.4.1 Espacio de observación ($\mathcal{S}$): Discretización del *lidar*

Procesar el vector completo de 360 valores del escáner láser incrementaría el tiempo de inferencia y la complejidad del modelo. Para simplificar la observación, se diseñó una discretización espacial del campo de visión en $n = 24$ sectores. De cada sector se extrae el valor mínimo observado (*min-pooling*), lo que ayuda a mantener la sensibilidad del agente ante obstáculos próximos.

Los valores se introducen en la red neuronal acotados por el rango operativo simulado de los sensores (3.5 m):

$$\mathbf{o}_t = [\min(L_1), \min(L_2), \dots, \min(L_{24})] \quad (4.1)$$

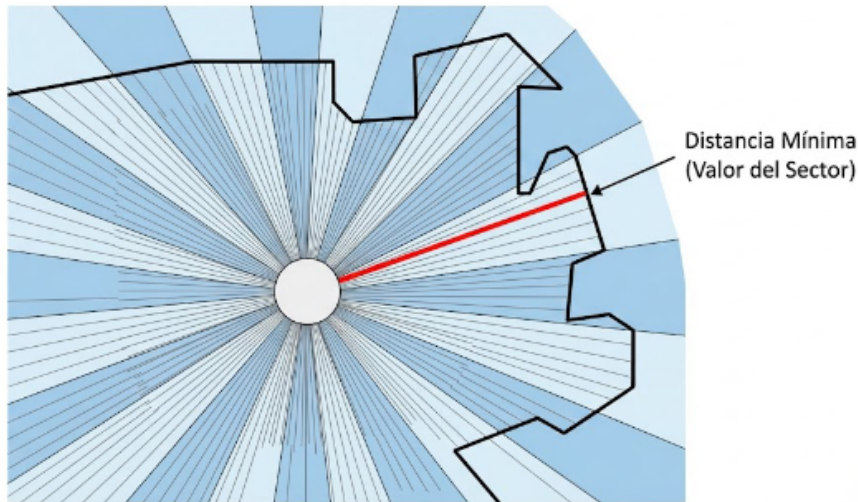


Figura 4.3: Reducción de la dimensionalidad del láser mediante agrupación por sectores.

4.4.2 Espacio de acciones ($\mathcal{A}$)

Para facilitar un control cinemático fluido, se optó por un espacio de acciones continuo (`spaces.Box`), permitiendo inferir comandos analógicos progresivos. Los límites espaciales se configuraron de la siguiente manera:

- **Velocidad lineal (v_x):** Se fijó un límite inferior estricto de 0.0 m/s para intentar suprimir comportamientos de retroceso infinito, donde el agente aprende a retroceder constantemente para evitar colisiones frontales. El límite superior se estructuró en dos configuraciones:
 - Fase de entrenamiento: Acotado a $[0.0, 0.22] \text{ m/s}$, correspondiente a la velocidad teórica nominal del TurtleBot3, buscando que la red aprenda maniobras de precisión.
 - Fase de despliegue: Ampliado a $[0.0, 0.50] \text{ m/s}$ para optimizar los tiempos de desplazamiento cuando la ruta está despejada.
- **Velocidad angular (ω_z):** Rango simétrico acotado a $[-1.5, 1.5] \text{ rad/s}$. Esta limitación persigue reducir giros excesivamente bruscos que podrían inducir error odométrico por deslizamiento lateral.

4.4.3 Función de diseño de la recompensa

La señal de recompensa orienta el proceso de optimización del agente. Se optó por una función de recompensa densa, que evalúa cualitativamente cada transición de estado basándose en los comandos cinemáticos (v_x, ω_z) y la distancia mínima percibida hacia los obstáculos (d_{min}):

$$\mathcal{R}_t = \begin{cases} -100.0 & \text{si } d_{min} < 0.15 \text{ (Colisión terminal)} \\ 2.0v_x - 0.5|\omega_z| + 0.2 & \text{si } d_{min} > 0.3 \text{ y } v_x > 0.15 \text{ (Bonificación)} \\ 2.0v_x - 0.5|\omega_z| & \text{en cualquier otro caso seguro} \end{cases} \quad (4.2)$$

Esta formulación persigue tres propósitos principales:

1. **Restricción de seguridad:** La penalización de -100.0 genera un gradiente negativo considerable, buscando que el modelo priorice evitar colisiones.
2. **Eficiencia cinemática:** El término base ($2.0v_x - 0.5|\omega_z|$) premia el avance rectilíneo mientras penaliza las oscilaciones angulares continuas.
3. **Mitigación del bloqueo:** La bonificación adicional de $+0.2$ busca incentivar al agente a mantener el avance frontal ($v_x > 0.15$) al atravesar zonas estrechas, intentando contrarrestar la precaución excesiva que suele generar estancamientos.

4.5 Estrategia de aprendizaje en dos fases

Dada la complejidad del espacio de estados y la estocasticidad de PPO, entrenar una red neuronal con inicialización aleatoria en un entorno con gran densidad de obstáculos puede

dificultar la convergencia. Para abordar este problema, se ha estructurado una metodología de aprendizaje progresivo, dividiendo la tarea en dos etapas de dificultad incremental:

- **Fase 1: Entrenamiento base:** En esta etapa, el agente se sitúa en un entorno controlado con baja densidad geométrica. El objetivo es que la red infiera la correlación básica entre las lecturas del *lidar* y las órdenes de velocidad. Durante aproximadamente 100.000 *timesteps*, se persigue que el agente aprenda a avanzar y evitar cualquier tipo de obstáculos. El resultado es un modelo inicial que evita movimientos erráticos aleatorios.
- **Fase 2: Entrenamiento especializado:** Una vez estabilizada la política de navegación básica, se transfieren los pesos a un escenario residencial de mayor complejidad. En esta fase se realiza un ajuste fino de la red neuronal. Se busca mejorar la reactividad del agente obligándolo a navegar con márgenes de seguridad más estrechos, lo cual es necesario para superar las situaciones que suelen paralizar a los algoritmos clásicos.

4.5.1 Mecanismo de reseteo del entorno

Al integrar ROS 2 con entornos de aprendizaje por refuerzo, pueden surgir desincronizaciones durante el reinicio de episodios. Para abordarlo, se programó una rutina de reseteo:

1. Publicación de comandos nulos ($v_x = 0, \omega_z = 0$) para frenar la simulación.
2. Llamada al servicio `/reset_simulation` seguida de un bloqueo temporal (ej. 2.0 segundos) para estabilizar el motor físico de Gazebo.
3. Verificación del búfer *lidar* para confirmar que las lecturas iniciales del nuevo episodio superan el umbral de impacto, previniendo falsos positivos por retardo en la red.

4.6 Diseño de la arquitectura híbrida

La delegación del control cinemático exclusivamente a una red neuronal (*end-to-end*) puede presentar dificultades para mantener el seguimiento de rutas a largo plazo. Para mitigar esta debilidad, se diseñó una arquitectura híbrida de control evaluada a 20 Hz. Este enfoque combina el guiado geométrico basado en odometría (control hacia la meta mediante *waypoints*) con la inferencia reactiva neuronal para la evasión.

Matemáticamente, el sistema se define como una función condicionada por la percepción de riesgo (d_{min}):

$$\begin{bmatrix} v_x \\ \omega_z \end{bmatrix} = \begin{cases} \begin{bmatrix} 0.0 \\ 2.0 \cdot \omega_{IA} \end{bmatrix} & \text{si } d_{min} < 0.25 \text{ (Evasi3n cr3tica)} \\ \begin{bmatrix} 0.10 \\ 1.5 \cdot \omega_{IA} \end{bmatrix} & \text{si } 0.25 \leq d_{min} < 0.45 \text{ (Evasi3n preventiva)} \\ \begin{bmatrix} 0.20 \\ 0.5 \cdot \omega_{IA} + 0.5 \cdot \omega_{odom} \end{bmatrix} & \text{si } 0.45 \leq d_{min} < 0.75 \text{ (Transici3n)} \\ \begin{bmatrix} 0.45 \\ \omega_{odom} \end{bmatrix} & \text{si } d_{min} \geq 0.75 \text{ (Odometr3a)} \end{cases} \quad (4.3)$$

Esta estructura permite que el robot mantenga el avance guiado hacia el destino cuando la ruta est3 despejada, cediendo gradualmente el control a las maniobras de esqui3a generadas por la inteligencia artificial conforme aumenta la proximidad de los obst3culos.

4.6.1 Sistema de recuperaci3n

Las din3micas topol3gicas pueden generar situaciones de estancamiento f3sico en m3nimos locales (por ejemplo, obst3culos en herradura). Para fomentar la autonom3a continua del robot, se incorpor3 un supervisor de estado o *watchdog*.

Si el robot detecta un bloqueo en la zona cr3tica ($d_{min} < 0.25 \text{ m}$) durante un n3mero determinado de ciclos de control (aprox. 65 iteraciones), el supervisor interrumpe temporalmente la salida de la red neuronal e inyecta una maniobra determinista: aplica una velocidad de retroceso ($v_x = -0.12 \text{ m/s}$) combinada con un giro ($\omega_z = 1.4 \text{ rad/s}$). Esta rutina persigue reubicar al chasis en una zona libre de colisi3n antes de devolver el control a la arquitectura principal.

4.7 Interfaz humano-robot (HRI)

Para dotar al sistema de operabilidad y seguimiento, se dise1n3 una interfaz gr3fica de usuario en Python utilizando Tkinter. Se seleccion3 esta librer3a por su ligereza computacional, reservando los recursos de la CPU para el algoritmo PPO y el simulador.

El principal reto de desarrollo fue gestionar la concurrencia entre procesos bloqueantes: tanto el nodo de ROS 2 (`rclpy.spin()`) como la interfaz gr3fica (`mainloop()`) requieren bucles principales. Para evitar que se bloqueen mutuamente, se implement3 una arquitectura as3ncrona mediante multihilo:

1. **Hilo secundario (*daemon thread*):** Se encarga de ejecutar el nodo de ROS 2 y la inferencia de la red PPO. Al asignarse como *daemon*, finaliza autom3ticamente al cerrar la aplicaci3n.
2. **Hilo principal (*main thread*):** Gestiona la renderizaci3n visual y las entradas del usuario de forma independiente.

Para sincronizar la visualización de los datos de telemetría provenientes del hilo secundario sin producir fallos de acceso concurrente, el hilo principal realiza un sondeo continuo cada 50 milisegundos para actualizar la interfaz.

5 Desarrollo

Este capítulo detalla la implementación técnica del *software* que compone el sistema de navegación autónoma. A lo largo de las siguientes secciones se describe el proceso de programación: la estructuración de los nodos en ROS 2, la gestión de la concurrencia asíncrona, el preprocesamiento sensorial en Python, la automatización del entrenamiento de la red neuronal y la codificación del controlador híbrido.

5.1 Arquitectura del sistema y flujo de datos en ROS 2

A nivel de programación, una de las consideraciones principales fue la integración de dos paradigmas de ejecución distintos. El *middleware* ROS 2 opera de forma asíncrona y reactiva mediante funciones de retrollamada (*callbacks*) activadas por eventos de red. Por el contrario, la librería Stable-Baselines3 (SB3) requiere un entorno de ejecución secuencial y síncrono, donde el tiempo de simulación avanza iterativamente en pasos discretos de control (*steps*).

Para abordar esta diferencia, el sistema se implementó utilizando el patrón de diseño *wrapper*. El código de ROS 2 actúa como enlace de bajo nivel con el simulador Gazebo, mientras que su lógica operativa se encapsula en una clase personalizada denominada *TurtleBotEnv*, la cual hereda de la interfaz estándar *gym.Env*.

El flujo de información y control de procesos se estructuró de la siguiente manera:

- **Adquisición perceptual:** El *script* principal inicializa un nodo de ROS 2 (`ppo_navegador_master`) suscrito a dos *topics*. Del *topic* `/scan` se recibe el vector de distancias del sensor *lidar*, entrada principal para la IA. Del *topic* `/odom` se extrae la pose espacial (x, y) y la orientación global. Dado que la odometría se transmite mediante cuaterniones espaciales, se programó una función de conversión (`euler_from_quaternion`) para obtener el *yaw* (rumbo), parámetro necesario para el cálculo trigonométrico hacia la meta. Se implementaron perfiles QoS *best effort* con un búfer de profundidad 10 para priorizar la lectura ágil del láser frente a posibles retardos de red.
- **Motor de inferencia y sincronización:** En el método `step()` del entorno, la red neuronal recibe el dato espacial preprocesado y devuelve el vector de acción inferido. Durante este cálculo, el bucle de ROS 2 captura nuevas lecturas mediante llamadas a `rclpy.spin_once(timeout_sec=0.01)`, evalúa la función de recompensa y actualiza el estado. Para buscar la sincronía entre el simulador y la red, se estableció un ciclo de control de 20 Hz mediante pausas de 0.05 segundos (`time.sleep(0.05)`).
- **Fusión y actuación:** La arquitectura híbrida, programada en la etapa de despliegue, evalúa la matriz de distancias. En función de la proximidad de los obstáculos, el sistema pondera la sugerencia de velocidad de la IA y el vector de ruta geométrica clásica, publicando el mensaje *twist* resultante en el *topic* `/cmd_vel` hacia los motores virtuales.

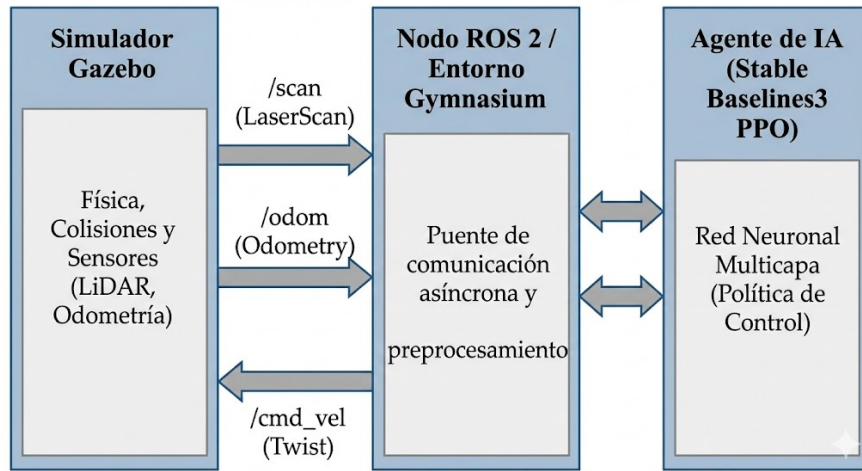


Figura 5.1: Diagrama de flujo de datos y paso de mensajes entre nodos en la arquitectura ROS 2 implementada.

5.2 Despliegue del plan de entrenamiento

La estrategia de aprendizaje en dos fases se automatizó mediante el *script* de Python (`entrenamiento_ppo_oficina.py`), utilizando la función `model.learn()` de SB3 y parametrizando el directorio de TensorBoard para el registro de métricas.

5.2.1 Fase 1: Entrenamiento base

La primera fase consistió en 100.000 iteraciones en el mapa inicial de simulación. Durante el desarrollo, se observó un problema de desincronización temporal: la latencia asíncrona de ROS 2 provocaba que, al reiniciar la simulación tras una colisión (`/reset_simulation`), el nuevo episodio heredara lecturas láser residuales del choque anterior.

Para evitar esta interferencia en la captura de datos, se desarrolló una rutina de reinicio en el método `reset()`. Esta función ejecuta secuencialmente:

1. Publicación de un *twist* con velocidades nulas para detener el chasis.
2. Invocación del servicio de reinicio físico de Gazebo con un tiempo máximo de espera (`wait_for_service(timeout_sec=2.0)`).
3. Ejecución de una espera activa para forzar el vaciado de la cola de mensajes del *topic* `/scan`. La inferencia se reanuda cuando el sistema verifica que la nueva distancia mínima percibida supera un umbral inicial ($d_{min} > 0.20\text{ m}$).

5.2.2 Fase 2: Entrenamiento en entorno complejo

Tras consolidar las primitivas de movimiento en la fase 1, se añadieron 50.000 iteraciones adicionales en el entorno residencial. En esta segunda etapa de aprendizaje se aplicó un ajuste

fino a los pesos neuronales. El objetivo metodológico consistió en exponer al agente a una geometría más restrictiva, con pasillos estrechos y giros pronunciados, forzando a la red a refinar la interpretación de las lecturas del *lidar* para navegar con márgenes de seguridad reducidos.

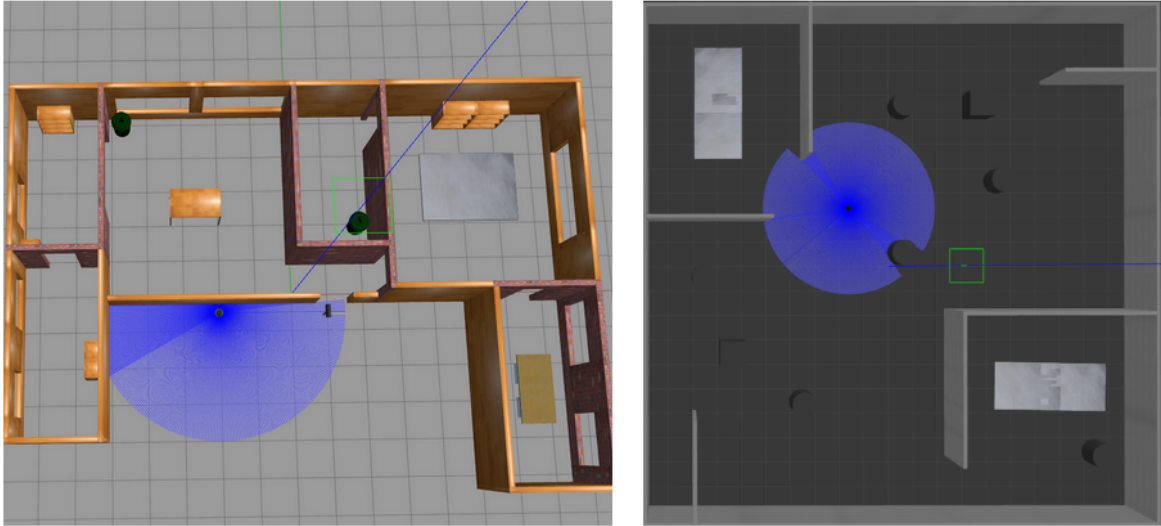


Figura 5.2: Escenarios de entrenamiento en Gazebo: Entorno residencial (izq.) para la fase 2 y entorno inicial (der.) para la fase 1.

5.3 Implementación del módulo de percepción y recompensa

5.3.1 Submuestreo *lidar* mediante *min-pooling*

El algoritmo de reducción de dimensionalidad se integró dentro de la retrollamada del escáner láser (Código 5.1). El *script* filtra valores infinitos (`math.isinf`) y nulos (`math.isnan`) generados por el simulador cuando el rayo no detecta colisiones, asignándoles el valor de rango máximo (3.5 metros).

A continuación, el algoritmo divide los 360 grados del escáner en 24 sectores equidistantes, aplicando la función `min()` a cada uno. El resultado se encapsula en un vector NumPy de 32 bits (`np.float32`) para facilitar su procesamiento en los tensores de PyTorch.

Código 5.1: Algoritmo de agrupación sectorial LiDAR implementado en el *callback* del entorno.

```
1 def scan_callback(self, msg):
2     ranges = msg.ranges
3     ranges = [r if not math.isinf(r) and not math.isnan(r) else 3.5 for r in ↵
4         ↵ ranges]
5     step = int(len(ranges) / self.n_sectores)
6
7     new_scan = []
8     for i in range(0, len(ranges), step):
9         sector = ranges[i:i+step]
```

```

9      new_scan.append(min(sector) if len(sector) > 0 else 3.5)
10
11     self.latest_scan = np.array(new_scan[:self.n_sectores], dtype=np.float32)
12     self.scan_ready = True

```

5.3.2 Codificación vectorizada de la recompensa

La formulación de las recompensa se implementó íntegramente en el método `step()` del *script* `entrenamiento_ppo_oficina.py`. Como se ilustra en el esquema de la figura 5.3.

En primer lugar, el código establece una restricción crítica de seguridad: si el láser detecta un obstáculo por debajo del umbral operativo ($\text{distancia_minima} < 0.15$), el sistema asigna una penalización de -100.0 puntos ($\text{reward} = -100.0$) y fuerza la finalización inmediata del episodio ($\text{done} = \text{True}$).

En los ciclos de simulación donde no se produce colisión, el algoritmo calcula un balance cinemático continuo para evaluar la eficiencia del movimiento. La instrucción $\text{reward} = (\text{twist.linear.x} * 2.0) - (\text{abs}(\text{twist.angular.z}) * 0.5)$ recompensa el avance frontal mientras penaliza los giros innecesarios, incentivando a la red a inferir trayectorias suaves.

Adicionalmente, se programó un bloque condicional para inyectar una bonificación ($\text{reward} += 0.2$) si el robot mantiene una velocidad lineal aceptable ($\text{twist.linear.x} > 0.15$) garantizando un margen libre superior a 0,3 metros ($\text{distancia_minima} > 0.3$). Esta adición en el código (*reward shaping*) busca mitigar el exceso de precaución de la IA, evitando que el agente frene a 0 m/s al atravesar pasillos estrechos.

Para posibilitar el análisis posterior, el entorno personalizado fue encapsulado utilizando la clase `Monitor` de la librería `stable_baselines3.common.monitor`. Esta envoltura (*wrapper*) registra automáticamente el histórico de las recompensas acumuladas y las exporta a un directorio local (`log_dir`), permitiendo procesar las gráficas de convergencia a través de la herramienta TensorBoard.

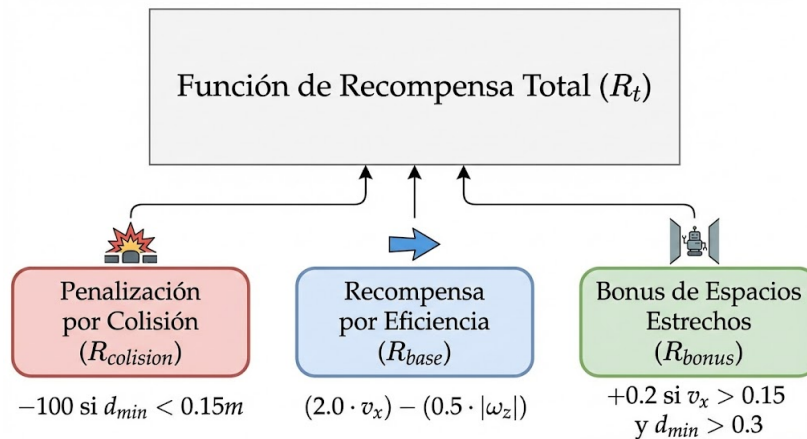


Figura 5.3: Esquema lógico y matemático de los componentes de la función de recompensa implementada en el código.

5.4 Programación de la arquitectura híbrida

La implementación del sistema de control se localiza principalmente en el método `step()` de la clase `TurtleBotEnv` dentro del *script* `probar_modelo.py`. Esta arquitectura híbrida se codificó mediante una estructura condicional `if-elif-else` que actúa como un selector de estados reactivo basado en la proximidad de obstáculos.

Para determinar la capa activa, el programa extrae el valor mínimo del vector sensorial (`distancia_minima = np.min(self.latest_scan)`) en cada ciclo de control. En función de este parámetro, se conmuta entre cuatro regímenes de actuación cinemática diseñados para equilibrar la eficiencia logística y la seguridad en la simulación (figura 5.4).

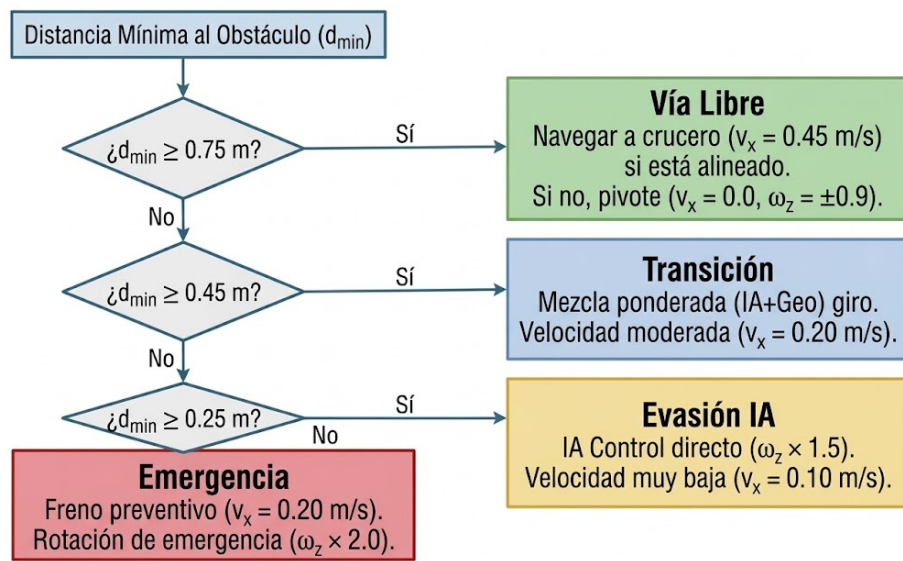


Figura 5.4: Esquema lógico implementado para la conmutación de capas del controlador híbrido.

El flujo operativo se implementó bajo las siguientes especificaciones técnicas:

- **Capa de vía libre** ($d_{min} \geq 0.75 \text{ m}$): Es el estado de crucero donde el control es puramente geométrico. Si el error angular calculado en `get_nav_logic()` supera los 0.6 rad , el código prioriza un giro de pivote ($\omega_z = \pm 0.9 \text{ rad/s}$) con velocidad lineal nula para encarar el objetivo. Una vez alineado, se aplica la velocidad máxima de 0.45 m/s .
- **Capa de transición** ($0.45 \leq d_{min} < 0.75 \text{ m}$): Se programó una mezcla ponderada para suavizar la entrada de la IA. El comando angular se calcula como la media aritmética entre la sugerencia de la red neuronal y el error de rumbo hacia el *waypoint* ($\text{twist.angular.z} = (\text{giro_ia} * 0.5) + (\text{error_angulo} * 0.5)$), manteniendo una velocidad moderada de 0.20 m/s .
- **Capa de evasión IA** ($0.25 \leq d_{min} < 0.45 \text{ m}$): El sistema otorga prioridad a la política aprendida. Se restringe la velocidad lineal a 0.10 m/s y se amplifica la respuesta de la

red neuronal aplicando un multiplicador de 1.5 sobre el giro (`giro_ia * 1.5`) para garantizar maniobras de esquivas proactivas.

- **Capa de emergencia** ($d_{min} < 0.25 \text{ m}$): Ante la proximidad crítica, se activa una rutina de frenado preventivo a 0.20 m/s . Para forzar una salida rápida de la zona de colisión, se satura el comando de rotación duplicando el valor inferido por la IA (`giro_ia * 2.0`).

5.4.1 Implementación del sistema de recuperación (*watchdog*)

Para gestionar situaciones de bloqueo donde la IA no logra resolver mínimos locales, se programó un supervisor algorítmico o *watchdog*. La lógica utiliza un acumulador (`self.contador_atasco`) que se incrementa en cada iteración siempre que d_{min} sea inferior a 0.25 m .

Si el contador alcanza el umbral de 65 ciclos (aproximadamente 3.25 segundos a la frecuencia operativa de 20 Hz), el código interrumpe el flujo normal e inyecta forzosamente una maniobra de rescate determinista: una velocidad de retroceso de -0.12 m/s y una rotación agresiva de 1.4 rad/s durante un intervalo de 0.6 segundos gestionado mediante `time.sleep()`. Tras esta secuencia, el contador se reinicia para permitir que el controlador híbrido retome el mando desde una posición con mayor visibilidad sensorial.

5.4.2 Maniobras de recuperación y planificación vectorial

A nivel de *software*, el seguimiento de la trayectoria geométrica en la capa libre se programó dentro de la función `get_nav_logic()`. Para determinar la acción de control, el código calcula continuamente el error de pose actual respecto a la meta espacial. La magnitud del error lineal se obtiene mediante la distancia euclídea plana (`math.sqrt(dx**2 + dy**2)`), mientras que la orientación de referencia se calcula utilizando la función arcotangente (`math.atan2(dy, dx)`). La implementación de `atan2` resuelve por código las ambigüedades de cuadrante, devolviendo un ángulo correctamente normalizado en el intervalo $[-\pi, \pi]$.

Para eliminar los bloqueos en mínimos locales durante la simulación, se programó un sistema de *watchdog* directamente en el método `step()` del entorno. La lógica opera evaluando una variable de estado (`self.contador_atasco`), la cual incrementa su valor si la lectura mínima del láser cae por debajo del umbral de los 0,25 metros.

Si la variable supera el valor de 65 iteraciones consecutivas (equivalente a 3,25 segundos de bloqueo a 20 Hz), el código entra en la condición de rescate: se interrumpe temporalmente la llamada a la red neuronal (`model.predict`) y se publica un objeto `Twist` predefinido en el tópico `/cmd_vel`. Este comando aplica una velocidad de retroceso de -0.12 m/s y una rotación de 1.4 rad/s durante 0,6 segundos mediante `time.sleep()`, con el objetivo de separar al chasis del obstáculo antes de continuar la ejecución normal del bucle.

Finalmente, la gestión de las rutas no se implementó como un envío directo de coordenadas, sino que se programó un enrutamiento estructurado mediante diccionarios en Python. Tal y como se ilustra en la figura 5.5, la variable `self.LUGARES` asocia cada destino con un *waypoint* de aproximación previo (por ejemplo, el nodo `P1_Pasillo` antes de entrar en `P1`). El método

`generar_ruta_dinamica()` desempaqueta este diccionario creando una máquina de estados secuencial.

Para gestionar la transición entre estos nodos intermedios, se estableció por código una tolerancia radial de 0,65 metros. Cuando el cálculo de la distancia euclídea en `get_nav_logic()` es inferior a este valor, el programa incrementa el índice del objetivo (`self.indice_obj += 1`) y el robot transiciona automáticamente hacia el siguiente *waypoint*. Esta implementación de código evita detenciones bruscas y genera un trazado fluido al atravesar zonas críticas.

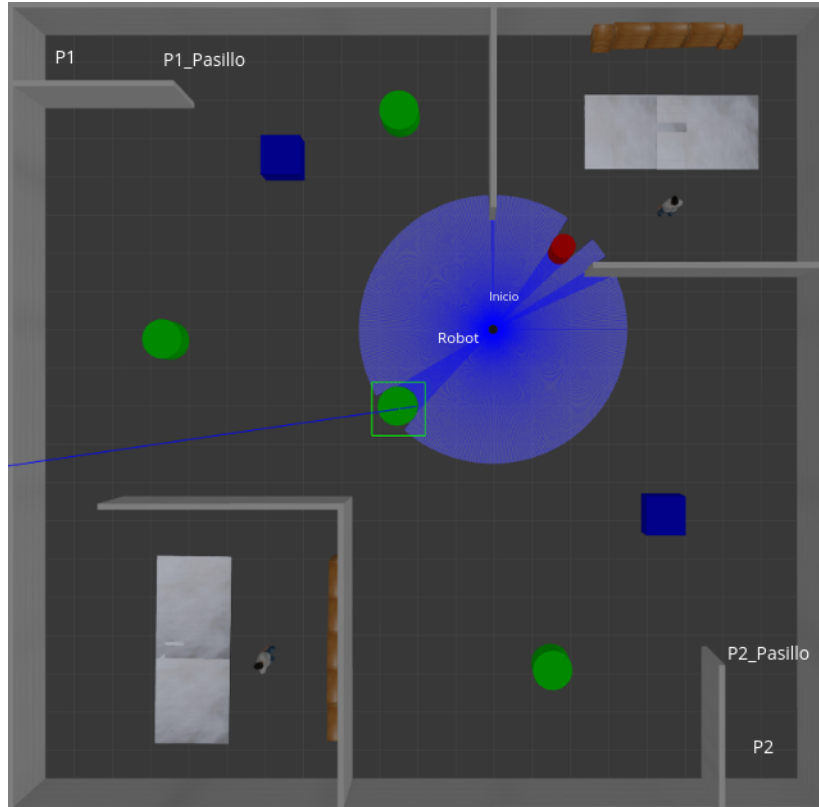


Figura 5.5: Visualización de la ruta programada utilizando *waypoints* de aproximación.

5.5 Evaluación ante un obstáculo dinámico simulado

Una vez obtenido el modelo ajustado (`modelo_finetuned`), se diseñó un escenario de evaluación en Gazebo para observar la respuesta del agente frente a elementos móviles no vistos durante el entrenamiento. En el *script* de prueba se integró un nodo que controla un obstáculo dinámico (cilindro rojo).

Este obstáculo es gestionado por la función `mover_obstaculo_callback`, la cual aplica velocidad lineal en el eje y para trazar una trayectoria perpendicular al avance del robot. Un temporizador (*timer*) invierte la dirección del actor ($v_y = \pm 0.4 \text{ m/s}$) cada 16.0 segundos, generando perturbaciones continuas en las lecturas frontales del escáner.

Esta prueba evalúa la capacidad de adaptación de la arquitectura híbrida en simulación. Aunque la red no fue entrenada con objetos en movimiento continuo, las variaciones en el vector de observación $\mathbf{o}_t$ forzaron al sistema a conmutar hacia las capas de evasión. La ejecución de esta prueba sugiere que el entrenamiento basado en DRL proporciona cierta capacidad de reacción ante obstáculos móviles imprevistos dentro del simulador.

5.6 Desarrollo de la interfaz gráfica

El proyecto incorpora una interfaz de usuario de escritorio programada con la librería Tkinter de Python (figura 5.6), la cual permite la parametrización de rutas y la monitorización de telemetría.

Uno de los aspectos a resolver en la implementación fue la gestión de procesos concurrentes bloqueantes. Tanto la interfaz visual (`root.mainloop()`) como el gestor de eventos de ROS 2 (`rclpy.spin()`) operan como bucles infinitos. Para evitar conflictos en la ejecución (*starvation*), se aplicó una arquitectura multihilo:

- **Hilo secundario (*daemon thread*):** Ejecuta en segundo plano el bucle de ROS 2 y la predicción del modelo PPO. Su naturaleza de *daemon* permite que el sistema finalice estos procesos al cerrar la aplicación.
- **Hilo principal (*main thread*):** Gestiona la interfaz de usuario, capturando eventos y refrescando los elementos visuales.



Figura 5.6: Interfaz gráfica operativa programada para la gestión y monitorización.

5.6.1 Sincronización de memoria

Dado que Tkinter no es seguro para operaciones asíncronas concurrentes , el hilo secundario no modifica la interfaz directamente para evitar errores de ejecución. En su lugar, el hilo principal ejecuta un bucle de sondeo: lee las variables globales de telemetría y actualiza los elementos de texto cada 50 milisegundos mediante `root.after(0, self._ui_upd)`. Este método permite un refresco de la información visual fluido sin generar interbloqueos.

6 Resultados

En este capítulo se exponen los resultados obtenidos tras la implementación del sistema de navegación autónoma en el entorno virtual. El análisis se fundamenta en datos empíricos recogidos durante las simulaciones: las métricas de entrenamiento extraídas de TensorBoard, los datos de rendimiento computacional y la telemetría del robot evaluada en misiones simuladas continuas.

El análisis se ha estructurado para evaluar el comportamiento del sistema en cuatro etapas: la influencia de la geometría del entorno en la percepción, la evolución del entrenamiento, la evaluación cuantitativa de la arquitectura híbrida, y las pruebas operativas en rutas logísticas simuladas.

6.1 Análisis de la percepción sensorial

Para evaluar la capacidad de adaptación de la red neuronal, se utilizó una estrategia de aprendizaje en dos fases sobre entornos con características de navegabilidad diferentes. La configuración de los obstáculos determina directamente la varianza de los datos que el sensor *lidar* inyecta en el espacio de observación.

Como se observa en la (figura 5.2), el entorno oficina presenta un espacio de estados geoméricamente estructurado. Los rayos del *lidar* impactan de forma perpendicular sobre paredes lisas y regulares, lo que reduce la dispersión de las mediciones y facilita que el agente identifique las distancias de seguridad durante las primeras iteraciones.

Por el contrario, el entorno residencial introduce una mayor complejidad topológica. Los obstáculos irregulares (como el mobiliario) provocan saltos de profundidad abruptos en las lecturas del escáner. Esto genera zonas de oclusión temporal que obligan al agente a desarrollar una política de evasión más conservadora. Esta diferencia estructural justifica la decisión metodológica de segmentar el entrenamiento, buscando una convergencia más estable del gradiente en la primera fase antes de introducir ruido geométrico.

A esta complejidad estática se sumó, en la fase de validación, la introducción de un obstáculo dinámico (simulación de tráfico humano). Su movimiento constante transversal a las rutas principales genera variaciones iterativas en las lecturas láser, permitiendo evaluar la capacidad de generalización y respuesta reactiva de la red neuronal en simulación.

6.2 Análisis de la evolución del entrenamiento PPO

Para evaluar el impacto del aprendizaje en dos fases, se han analizado las métricas internas de la red *actor-critic*, desglosando el comportamiento de las principales variables del algoritmo

PPO. En la (tabla 6.1) se recogen los valores en tres hitos clave del proceso, comparando el progreso del modelo base con el modelo ajustado en el entorno residencial.

	Hito: 10.000 Pasos		Hito: 30.000 Pasos		Hito: 50.000 Pasos	
Métricas PPO	Oficina	Casa	Oficina	Casa	Oficina	Casa
Long. Supervivencia (<code>ep_len_mean</code>)	13,3	883,0	929,0	1.230,0	929,0	1.410,0
Recompensa Media (<code>ep_rew_mean</code>)	-96,1	+196,0	+201,0	+349,0	+201,0	+445,0
Varianza Explicada (<code>explained_var</code>)	0,674	0,441	0,794	0,661	0,695	0,726
Error Predictivo (<code>value_loss</code>)	0,631	77,8	0,383	41,6	0,097	124,0
Pérdida de Entropía (<code>entropy_loss</code>)	-2,360	-1,880	-1,540	-1,750	-0,645	-1,400

Tabla 6.1: Tabla comparativa de las métricas de la red neuronal en las 2 fases.

6.2.1 Convergencia de la política: Recompensa y supervivencia

Al analizar el rendimiento general del agente (figura 6.1), se observa una correlación morfológica directa entre las curvas de recompensa media (azul) y duración del episodio (verde). Esta similitud gráfica sugiere que el agente incrementa su retorno escalar prolongando su tiempo de supervivencia simulada y evitando el estado terminal de colisión.

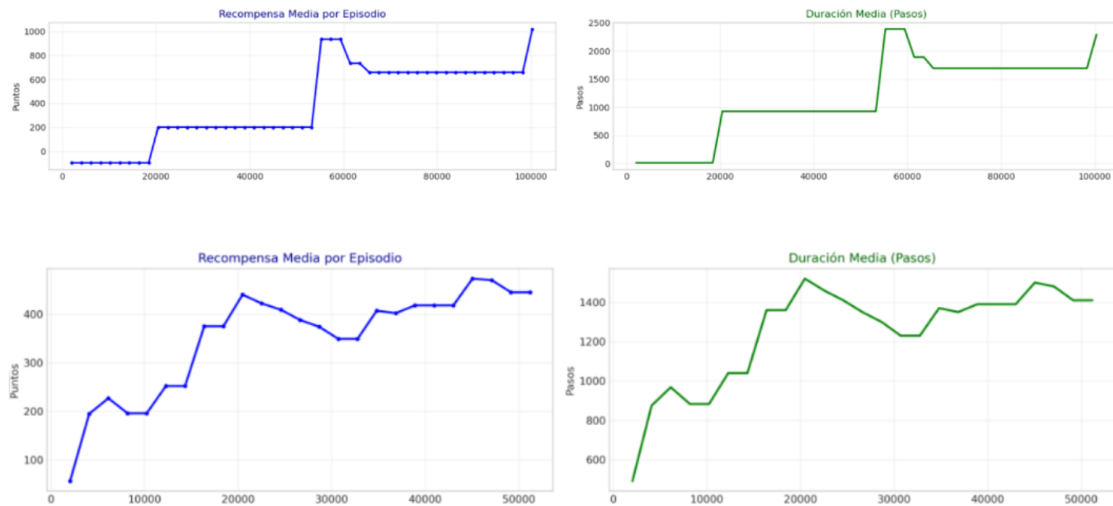


Figura 6.1: Comparativa de la duración media y recompensa del agente.

En la fase 1 (gráficas superiores), el modelo parte sin conocimiento previo, lo que resulta en una baja supervivencia inicial. La curva presenta un comportamiento escalonado típico en algoritmos de gradiente de política: el agente se mantiene en políticas subóptimas hasta que la exploración estocástica le permite descubrir secuencias más eficaces, estabilizándose de forma progresiva.

El efecto de la transferencia de conocimiento se evidencia en la fase 2 (gráficas inferiores). A pesar del notable incremento en la complejidad geométrica del mapa, el agente inicia el *fine-tuning* con una tasa de supervivencia base elevada (arrancando en casi 500 pasos y escalando rápidamente). Se observa una caída transitoria en el rendimiento alrededor de la iteración 30.000. Este valle suele representar un periodo de reajuste donde el modelo adapta su política al interactuar con el mobiliario no estructurado. Tras este periodo, la red muestra capacidad de adaptación, superando los 1400 pasos medios de supervivencia antes de la iteración 50.000.

6.2.2 Exploración vs. explotación: Análisis de entropía

La métrica de entropía indica el nivel de estocasticidad en la toma de decisiones del agente. En la implementación de Stable-Baselines3, una tendencia ascendente en la pérdida de entropía (acercándose a cero) implica una disminución de la entropía absoluta, traduciéndose en una política más determinista o enfocada a la explotación.

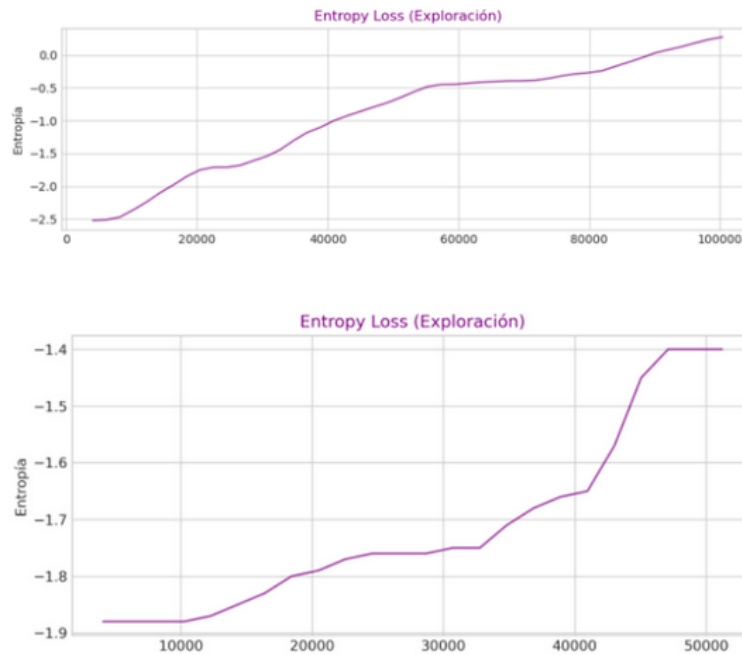


Figura 6.2: Comparativa de la entropía. Arriba: Oficina. Abajo: Casa.

En la fase 1 (gráfica superior), se observa una tendencia ascendente constante. Al tratarse de un entorno más predecible geométricamente, la red reduce su tasa de exploración de forma ininterrumpida, convergiendo hacia valores de pérdida cercanos a cero en el tramo final, lo que sugiere un comportamiento determinista.

En contraste, el comportamiento durante la fase 2 (gráfica inferior) refleja la complejidad del entorno residencial. En las primeras 12.000 iteraciones, la curva se mantiene plana, indicando que el agente mantiene la exploración activa al enfrentarse a oclusiones y obstáculos.

los irregulares nuevos. Posteriormente, la entropía se estabiliza en un umbral más negativo (-1,400). Este dato sugiere que el modelo ha preservado un grado de aleatoriedad en sus acciones, lo que constituye una posible estrategia aprendida para intentar evitar mínimos locales o bloqueos topológicos entre el mobiliario.

6.2.3 Estabilidad predictiva: Varianza y error de la red crítica

El comportamiento del motor cognitivo se completa analizando la red *critic*, evaluada mediante el error predictivo y la varianza explicada.

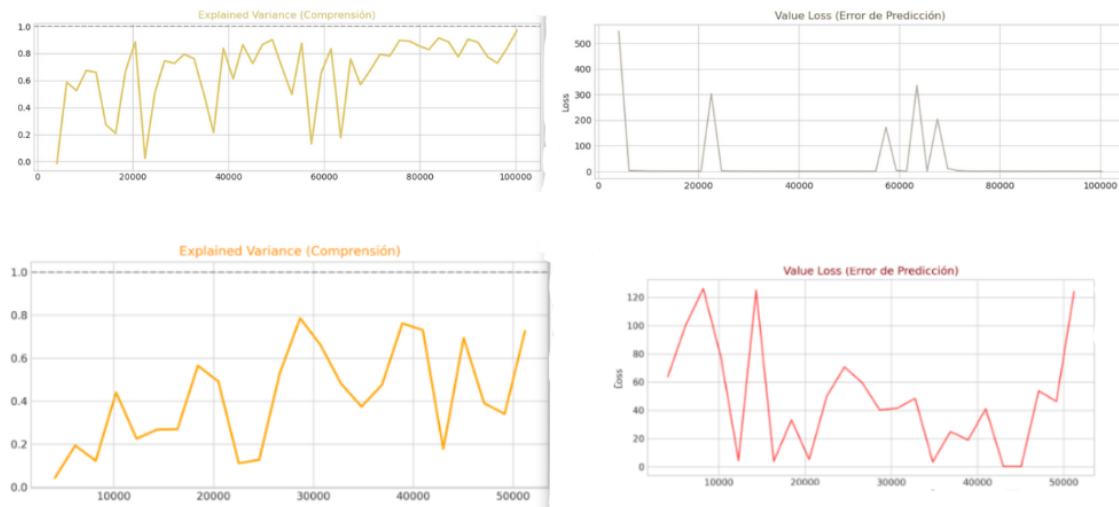


Figura 6.3: Comparativa de la varianza y el error predictivo.

En la fase 1 (gráficas superiores), el error de predicción desciende tras un pico inicial y se mantiene en valores próximos a cero. De forma correlacionada, la varianza explicada se estabiliza en el rango de 0,8 a 1,0. Esto indica que, dada la regularidad del entorno de oficina, el crítico logra aproximar el modelo del entorno y anticipar las recompensas futuras con mayor precisión.

Por el contrario, en la fase 2 (gráficas inferiores) se evidencia una mayor dificultad de estimación. El error predictivo muestra una alta volatilidad, oscilando continuamente, y la varianza explicada sufre caídas periódicas. Este comportamiento ruidoso suele derivar de las oclusiones dinámicas provocadas por el mobiliario. Sin embargo, el hecho de que la política de control (red *actor*) mantenga altas tasas de supervivencia, a pesar de las variaciones predictivas del crítico, apoya la utilidad del mecanismo de recorte (*clipping*) de PPO. Este límite estructural evita actualizaciones perjudiciales en la política de navegación ante estimaciones ruidosas.

6.2.4 Rendimiento computacional y eficiencia temporal

El entrenamiento de modelos en simuladores físicos tridimensionales suele requerir recursos computacionales considerables. El análisis de los tiempos de ejecución indica un rendimiento

estable durante todo el proceso.

La fase 1 (100.352 *timesteps*) se completó en aproximadamente 1 hora y 41 minutos. Por su parte, la fase 2 (51.200 *timesteps*) requirió 52 minutos. Esto supone un tiempo de entrenamiento acotado en comparación con las etapas necesarias en los enfoques de visión por computador.

Durante ambas etapas, la tasa de procesamiento se mantuvo en una media de 16 FPS. Mantener este flujo iterativo implica que la sobrecarga del sistema fue reducida a pesar de la concurrencia de procesos entre las dinámicas de Gazebo, las comunicaciones de ROS 2 y los cálculos tensoriales de PyTorch. Los datos observados sugieren que los mecanismos de sincronización implementados permitieron la ejecución del aprendizaje sin introducir penalizaciones temporales significativas.

6.3 Evaluación cuantitativa de la arquitectura híbrida

Para evaluar el comportamiento de la arquitectura híbrida, se sometió al algoritmo a una batería de 20 pruebas en simulación. En cada misión, el robot debía alcanzar diferentes coordenadas navegando entre obstáculos estáticos e intersectando la trayectoria del obstáculo dinámico.

El análisis comparativo, detallado en la (tabla 6.2), evalúa la tasa de éxito (llegada a meta), el porcentaje de atascos (congelación que requiere la activación del *watchdog*) y la media aritmética de colisiones por misión simulada.

Configuración de Control	Tasa de Éxito (%)	Atascos (%)	Media de Colisiones
Solo navegación geométrica	0 %	100 %	4,0
Solo IA (aprendizaje por refuerzo)	60 %	40 %	1,0
Arquitectura híbrida	90 %	10 %	0,1

Tabla 6.2: Evaluación simulada basada en 20 pruebas con las 3 métricas de rendimiento.

El análisis de estas métricas sugiere ventajas operativas al utilizar el esquema jerárquico propuesto. Al operar exclusivamente con navegación geométrica clásica, se observa una tasa de éxito nula y una media de 4,0 colisiones por intento, derivando frecuentemente en bloqueos frente al obstáculo dinámico. Por otro lado, delegar el control de forma exclusiva a la IA reduce los impactos directos (1,0 colisión de media), pero incrementa los problemas de orientación a largo plazo (40 % de atascos).

La arquitectura híbrida mostró un balance operativo más adecuado en estas simulaciones, alcanzando un 90 % de éxito y limitando la media de colisiones a 0,1. Esta configuración fundamenta su comportamiento en una transición de cuatro niveles guiada por la distancia de seguridad (d_{min}).

6.4 Validación en misión secuencial

La validación operativa final consistió en someter al sistema a un ciclo logístico simulado continuo (Base $\rightarrow$ Destino 1 $\rightarrow$ Destino 2 $\rightarrow$ Base) manteniendo activo el obstáculo dinámico. Tras la ejecución de 10 ciclos en simulación, se extrajeron las siguientes observaciones:

- **Tasa de completitud:** El recorrido se finalizó con éxito en el 90 % de las ocasiones (9 de 10 intentos), resolviendo las intersecciones y retornando al origen autónomamente.
- **Tiempos de ciclo:** La duración media del recorrido fue de 264,6 segundos, con una desviación de $\pm 5,1$ segundos. Este estrecho margen sugiere consistencia en las trayectorias trazadas e indica que la conmutación de los algoritmos de evasión no supone una penalización temporal desmedida.
- **Estabilidad multihilo:** Se observó una actualización fluida en la interfaz gráfica, lo que apoya la viabilidad del diseño concurrente para manejar la carga de procesamiento del *middleware* y la red neuronal en paralelo.

6.5 Análisis de eficiencia logística y consumo simulado

Para aproximar la viabilidad operativa teórica del sistema, se incluyó una estimación de consumo energético basada en la telemetría recopilada durante la simulación. La (tabla 6.3) desglosa el tiempo invertido y el coste porcentual teórico asociado a cada tramo.

Tramo logístico simulado	Distancia Aprox.	Tiempo Medio (s)	Consumo Est. (%)
Base $\rightarrow$ Destino 1	11,9 m	90,2 s	8,1 %
Operación descarga (H1)	0,0 m	2,5 s	0,5 %
Destino 1 $\rightarrow$ Destino 2	22,6 m	110,3 s	12,2 %
Operación descarga (H2)	0,0 m	2,5 s	0,4 %
Destino 2 $\rightarrow$ Base	12,5 m	59,1 s	6,8 %
Total ciclo completo	$\approx 47,0$ m	264,6 s	28,0 %

Tabla 6.3: Desglose de tiempos y estimación de consumo energético en simulación.

Se observa que el tránsito entre el destino 1 y el destino 2 resulta ser el más demandante en términos temporales y energéticos. Esta ruta cruza la intersección central del mapa, provocando que el sistema conmute reiteradamente hacia las capas de evasión para sortear al obstáculo, lo que conlleva una reducción en la velocidad media de avance.

El retorno a la base, por el contrario, presenta una mayor eficiencia teórica. La menor densidad geométrica en ese cuadrante permite al controlador operar mayoritariamente en la capa libre. El consumo simulado por ciclo total asciende al 28,0 %, lo que sugeriría que, bajo

condiciones ideales equivalentes a las de la simulación, la plataforma podría ejecutar aproximadamente 3 rutas logísticas completas manteniendo un margen de seguridad razonable. Estos resultados apoyan la viabilidad técnica de la arquitectura propuesta dentro del entorno evaluado.

7 Conclusiones

7.1 Conclusiones finales

Este trabajo fin de grado ha abordado el estudio y la implementación de distintas arquitecturas de control robótico, partiendo de los fundamentos geométricos clásicos hasta llegar a la aplicación de algoritmos de aprendizaje por refuerzo profundo. Tras analizar los datos recopilados en la fase de simulación y evaluar el comportamiento del agente, se extraen las siguientes observaciones empíricas sobre la viabilidad de estas metodologías en entornos virtuales.

En primer lugar, se ha observado que los algoritmos de planificación local tradicionales, aunque computacionalmente ligeros y estables en entornos estáticos, presentan limitaciones operativas frente a obstáculos en movimiento. La validación en simulación sugirió que enfoques como la ventana dinámica (DWA) pueden derivar en el bloqueo del robot (fenómeno del *freezing robot*) al no hallar trayectorias libres de colisión a corto plazo. Frente a esto, el despliegue del algoritmo PPO ha supuesto una alternativa de control viable. En los escenarios evaluados, la red neuronal ha mostrado capacidad para inferir estrategias de evasión proactivas, anticipando parcialmente la trayectoria de los obstáculos y ejecutando maniobras de adaptación que un algoritmo puramente determinista requeriría programar mediante reglas explícitas.

Por otro lado, los ensayos indican que delegar la totalidad del control de navegación a un modelo neuronal (*end-to-end*) puede generar sistemas menos predecibles a largo plazo respecto a la consecución del objetivo. En este punto se fundamenta la propuesta de la arquitectura híbrida desarrollada en este trabajo. Al utilizar la odometría geométrica para guiar al robot hacia su meta e intervenir con la IA de forma reactiva, se ha buscado un balance operativo. El robot mantiene el rumbo general hacia su objetivo, apoyándose en el ajuste fino únicamente para esquivar imprevistos al rebasar el umbral de seguridad.

A nivel de integración de *software*, el proyecto ha puesto de manifiesto los requerimientos técnicos para sincronizar ecosistemas de distinta naturaleza. Compatibilizar el flujo de datos asíncrono de ROS 2 Humble con la ejecución síncrona de Stable-Baselines3 ha requerido una gestión estricta de la concurrencia de procesos. Asimismo, se comprobó en la simulación que el aprendizaje en dos fases y la reducción dimensional del vector *lidar* mediante *min-pooling* ayudan a mitigar el sobreajuste del modelo durante el entrenamiento. Esta compresión de datos facilita que la inferencia de la red mantenga un bajo coste computacional, lo cual constituye un requisito previo para una eventual ejecución en el *hardware* de un TurtleBot3 físico.

En conclusión, los resultados obtenidos en el simulador sugieren que el aprendizaje por refuerzo representa una herramienta técnica adaptable para la robótica móvil, especialmente

cuando se integra dentro de una arquitectura de control estructurada que complemente los métodos analíticos de la ingeniería clásica.

7.2 Líneas de trabajo futuro

La robótica autónoma es un campo en evolución continua, y el sistema desarrollado en simulación sienta una base metodológica sobre la cual es posible plantear nuevas líneas de investigación. Atendiendo a la progresión del proyecto, se proponen las siguientes continuaciones a corto y medio plazo:

- **Transferencia *sim-to-real*:** Un paso metodológico directo consistiría en la transferencia del modelo neuronal al chasis físico del TurtleBot3 Burger. Aunque Gazebo recrea la cinemática de forma aproximada, probar el modelo en un entorno real introduciría variables físicas no procesadas previamente por la IA, como el ruido térmico de los sensores, el deslizamiento asimétrico por la fricción del suelo o la latencia real de la red de comunicaciones. Evaluar la resiliencia de la política neuronal ante estas perturbaciones constituiría un área de trabajo relevante.
 - **Ampliación sensorial y visión computacional:** La percepción del entorno podría expandirse. Actualmente, el agente toma decisiones basándose exclusivamente en un escáner *lidar* bidimensional, lo que limita su detección frente a obstáculos que no corten el plano del láser (como voladizos o elementos suspendidos). Integrar una cámara de profundidad (RGB-D) y sustituir la arquitectura de perceptrón multicapa por redes neuronales convolucionales (CNN) permitiría al agente procesar nubes de puntos en 3D, ampliando su volumen de detección.
 - **Navegación social frente a obstáculos dinámicos:** En el ámbito algorítmico, cabría la posibilidad de modificar la función de recompensa. En la versión actual, el agente trata al obstáculo dinámico humanoide como un obstáculo cilíndrico estándar. Una línea de mejora consistiría en penalizar las invasiones del espacio personal humano, buscando que la red aprenda reglas de navegación social (como ceder el paso o realizar adelantamientos manteniendo distancias de confort), lo que facilitaría la integración de estos robots en entornos públicos.
 - **Sistemas multiagente (MARL):** Finalmente, el enfoque podría escalarse hacia un escenario multiagente. Entrenar a una flota de robots para que operen simultáneamente en el mismo espacio de trabajo permitiría explorar cómo los agentes controlados por IA pueden interactuar y coordinar sus rutas de forma descentralizada para evitar bloqueos mutuos, una aplicación de interés directo en la automatización de almacenes logísticos e industriales.
-

Bibliografía

- [1] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*. MIT Press, 2nd ed., 2011.
- [2] Mint for People, “Robots móviles autónomos: ¿qué son y para qué sirven?.” <https://mintforpeople.com/noticias/robots-moviles-autonomos/>, 2024.
- [3] B. Siciliano, L. Sciavicco, L. Villani, and G. Oriolo, *Robotics: Modelling, Planning and Control*. Springer, 2009.
- [4] E. W. Dijkstra, “A note on two problems in connexion with graphs,” *Numerische Mathematik*, vol. 1, pp. 269–271, 1959.
- [5] M. Seder and I. Petrovic, “Local path planning of autonomous mobile robots based on the dynamic window approach,” in *2007 IEEE International Symposium on Industrial Electronics*, 2007.
- [6] Z. Haque, M. Zubair, and M. Atiquzzaman, “Deep reinforcement learning for autonomous vehicle control: A survey,” *IEEE Access*, vol. 10, pp. 125345–125372, 2022.
- [7] R. S. Sutton and A. G. Barto, *Reinforcement learning: An introduction*. MIT press, 2nd ed., 2018.
- [8] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” *arXiv preprint arXiv:1707.06347*, 2017.
- [9] ROBOTIS, *TurtleBot3 e-Manual*, 2024.
- [10] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot operating system 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, 2022.
- [11] N. Koenig and A. Howard, “Design and use paradigms for gazebo, an open-source multi-robot simulator,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2004.
- [12] J. Tobin *et al.*, “Domain randomization for transferring deep neural networks from simulation to the real world,” in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 23–30, 2017.
- [13] MathWorks, “What is slam (simultaneous localization and mapping)?.” <https://es.mathworks.com/discovery/slam.html>, 2024. Accedido en 2024.

-
- [14] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
 - [15] J. Borenstein and Y. Koren, "The vector field histogram-fast obstacle avoidance for mobile robots," *IEEE transactions on robotics and automation*, vol. 7, no. 3, pp. 278–288, 1991.
 - [16] D. Fox, W. Burgard, and S. Thrun, "The dynamic window approach to collision avoidance," *IEEE Robotics & Automation Magazine*, vol. 4, no. 1, pp. 23–33, 1997.
 - [17] Amazon Web Services (AWS), "¿qué es una red neuronal artificial?," <https://aws.amazon.com/es/what-is/neural-network/>, 2024. Accedido en 2024.
 - [18] V. Mnih *et al.*, "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
 - [19] V. Mnih *et al.*, "Playing atari with deep reinforcement learning," *arXiv preprint arXiv:1312.5602*, 2013.
 - [20] G. Brockman *et al.*, "Openai gym," *arXiv preprint arXiv:1606.01540*, 2016.
 - [21] A. Raffin *et al.*, "Stable-baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021.
 - [22] K. Zhu and T. Zhang, "Deep reinforcement learning based mobile robot navigation: A review," *Tsinghua Science and Technology*, vol. 26, no. 5, pp. 674–691, 2021.
 - [23] R. Brooks, "A robust layered control system for a mobile robot," *IEEE journal of robotics and automation*, vol. 2, no. 1, pp. 14–23, 1986.
 - [24] A. Stentz, "Optimal and efficient path planning for partially-known environments," in *Proceedings of the 1994 IEEE International Conference on Robotics and Automation*, pp. 3310–3317, 1994.
 - [25] A. Paszke *et al.*, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, pp. 8024–8035, 2019.
 - [26] L. Tai, G. Paolo, and M. Liu, "Virtual-to-real deep reinforcement learning: Continuous control of mobile robots for mapless navigation," in *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 31–36, 2017.
 - [27] T. P. Lillicrap *et al.*, "Continuous control with deep reinforcement learning," *arXiv preprint arXiv:1509.02971*, 2015.
 - [28] T. Haarnoja *et al.*, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *International Conference on Machine Learning (ICML)*, pp. 1861–1870, 2018.
-